

Abdul Wahid

CND Lab Report (Volume 1 of 3)

EC-Council CND (Certified Network Defender)

Data Security – Encryption at Rest (VeraCrypt, TDE & Always Encrypted)

Data Encryption at Rest & SQL Server Security

In this part of Module 10, I successfully completed the lab environment setup and the first three critical labs focused on **Data at Rest Encryption**.

What was covered:

- Configured the VMware virtual network environment (VMnet1 Host-only + VMnet3 NAT)
- Prepared four virtual machines (Server-1, Web Server, Finance-Dept, and AdminMachine-1)
- Implemented **VeraCrypt** for file-level encryption
- Enabled **Transparent Data Encryption (TDE)** on SQL Server
- Applied **Always Encrypted** for column-level protection in SQL Server

This section focuses on protecting data while it is stored on the disk, one of the most important layers of modern cybersecurity.

Table of Contents

Virtual Network Editor Setting 4

Network Setup for VM's 5

Encrypting Data at Rest Using VeraCrypt 7

Download and install VeraCrypt. 8

Create Encrypted File Container 10

Set Encryption Options (AES) & Volume Size 12

Mount the Encrypted Volume 14

Unmount the Volume 16

Verify Encryption (Container becomes unreadable) 17

Remount & Confirm Data Integrity 18

Implementing Transparent Data Encryption (TDE) on SQL Server 20

Download & Install SQL Server Developer Edition 21

Install SQL Server Management Studio (SSMS) 29

Create Database Master Key 36

Create Certificate 37

Create Database Encryption Key 39

Enable Transparent Data Encryption (TDE) 39

Implementing Always Encrypted in SQL Server 42

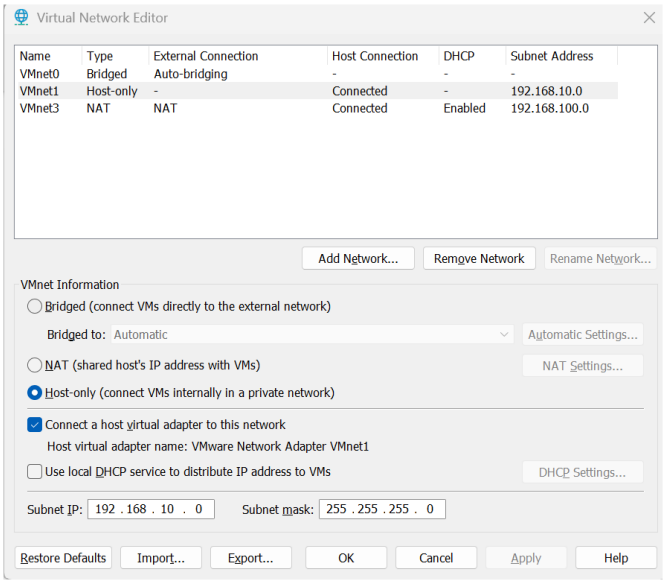
Create Column Master Key 45

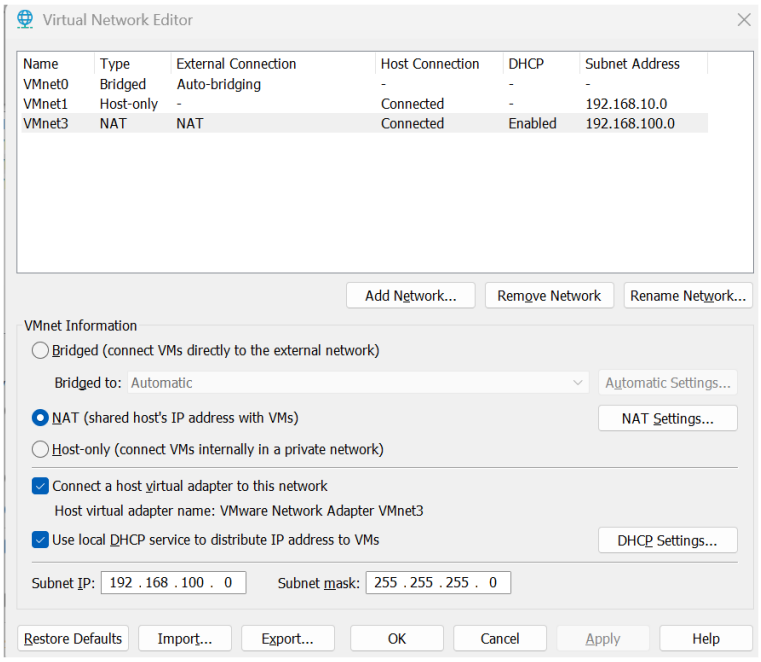
Verify Column is Encrypted (Data appears unreadable) 48

Conclusion – Module 10 (Part 1/3) 50

Virtual Network Editor Setting

Network	Type	Subnet	DHCP	Purpose
VMnet1	Host-only	192.168.10.0/24	Disabled (manual/static)	Internal isolated segment (If needed)
VMnet3	NAT	192.168.100.0/24	Enabled	Internet access (tool downloads: VeraCrypt, SQL Server, Gpg4win, AOMEI, EaseUS, Kernel, MiniTool, Disk Drill)

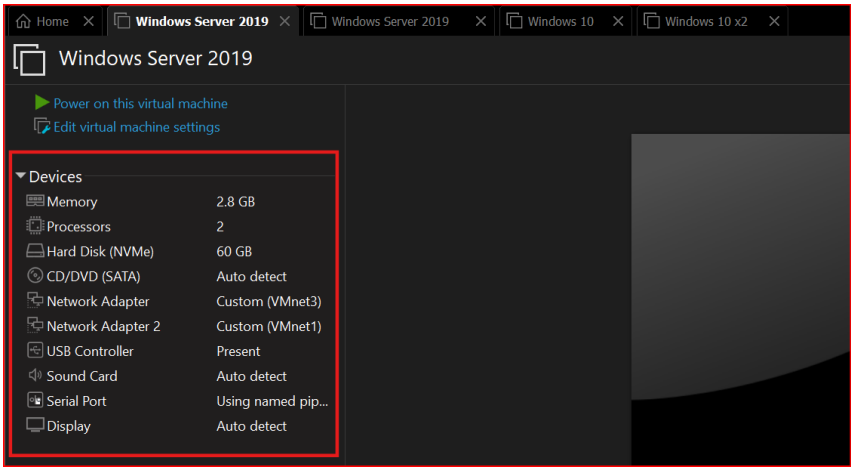




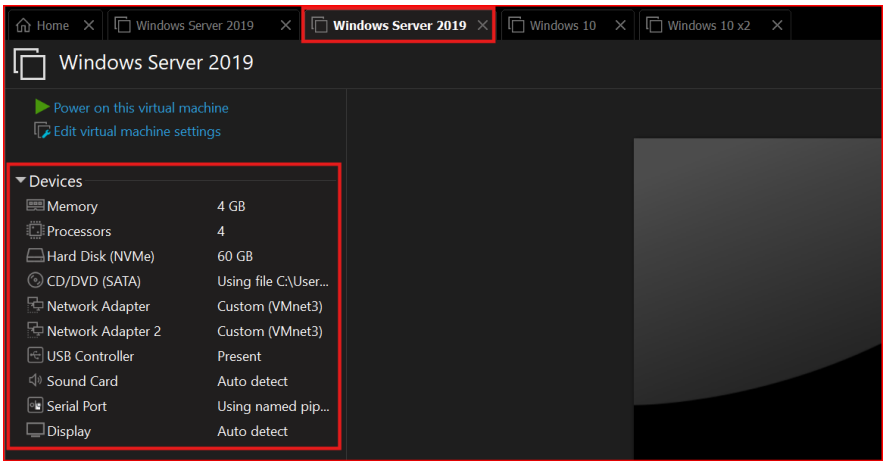
Network Setup for VM's

VM	Role (Module 10)	OS	RAM	CPU	Disk	NIC 1	NIC 2	Labs Used
Server-1	WAMP/WordPress host for SSL lab	Windows Server 2019	2.8 GB	2	60 GB	VMnet3 (NAT)	VMnet1 (Host-only)	Lab 4
Web Server	SQL Server + Gpg4win	Windows Server 2019	4 GB	4	60 GB	VMnet3 (NAT)	VMnet3 (NAT)	Labs 2, 3, 5
Finance-Dept	VeraCrypt client	Windows 10	2.5 GB	2	50 GB	VMnet1 (Host-only)	VMnet3 (NAT)	Lab 1
AdminMachine-1	Backup/Recovery/BitLocker	Windows 10 x2	2.5 GB	2	50 GB	VMnet1 (Host-only)	VMnet3 (NAT)	Labs 6,7,8,9,10,11

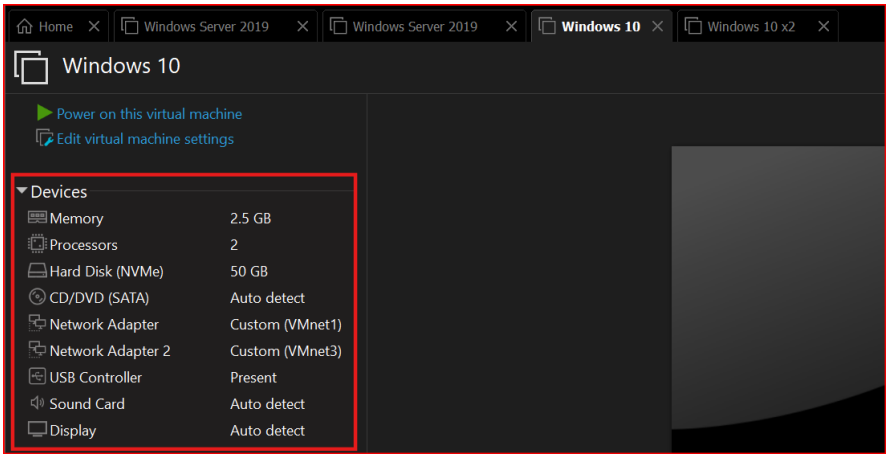
Server-1



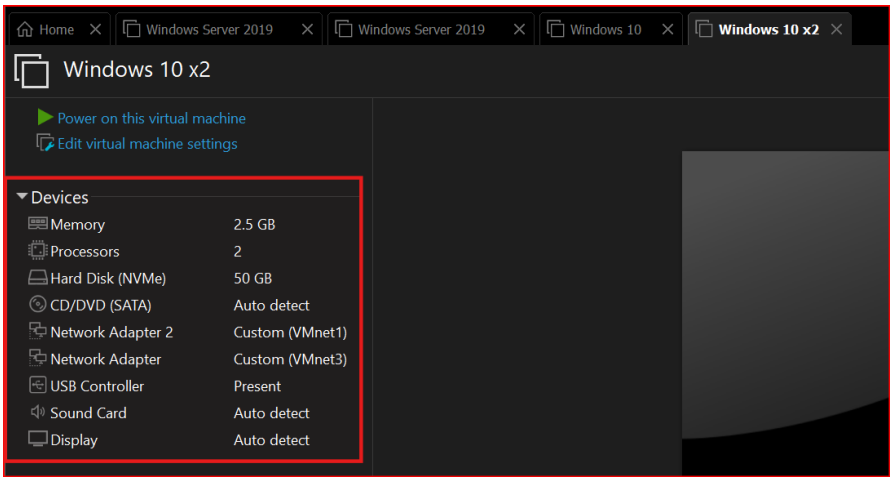
Server-2



Admin Machine



Finance Machine



Encrypting Data at Rest Using VeraCrypt

Summary: This lab teaches how to protect data at rest by creating an encrypted virtual disk (container) using VeraCrypt — a free, open-source on-the-fly encryption tool. The container behaves like a normal drive once mounted, but its contents stay encrypted whenever it's unmounted.

Objective: Install VeraCrypt on the Finance-Dept machine, create an encrypted volume, mount it, and verify that data stored inside it is protected.

Key Result: When unmounted, the container appears as meaningless encrypted data. The contents can only be accessed after mounting it with the correct password. This demonstrates how VeraCrypt creates a secure "locked box" for sensitive files.



Turn on Finance-Dept VM and log in.

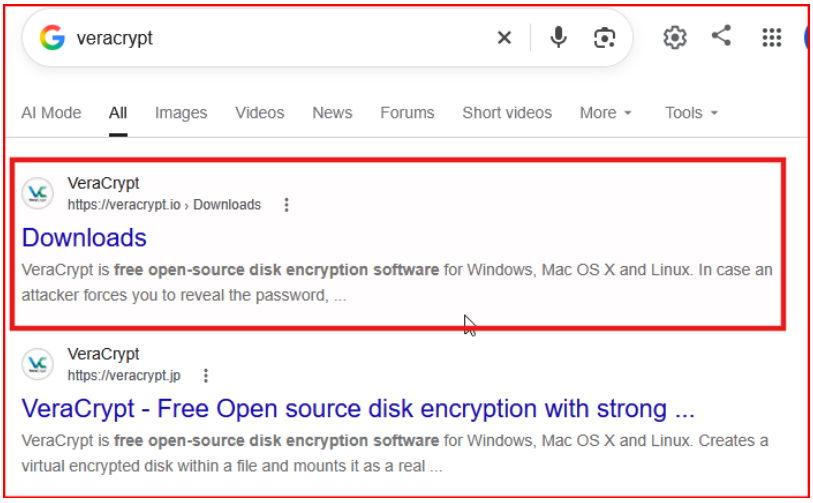
Network Check

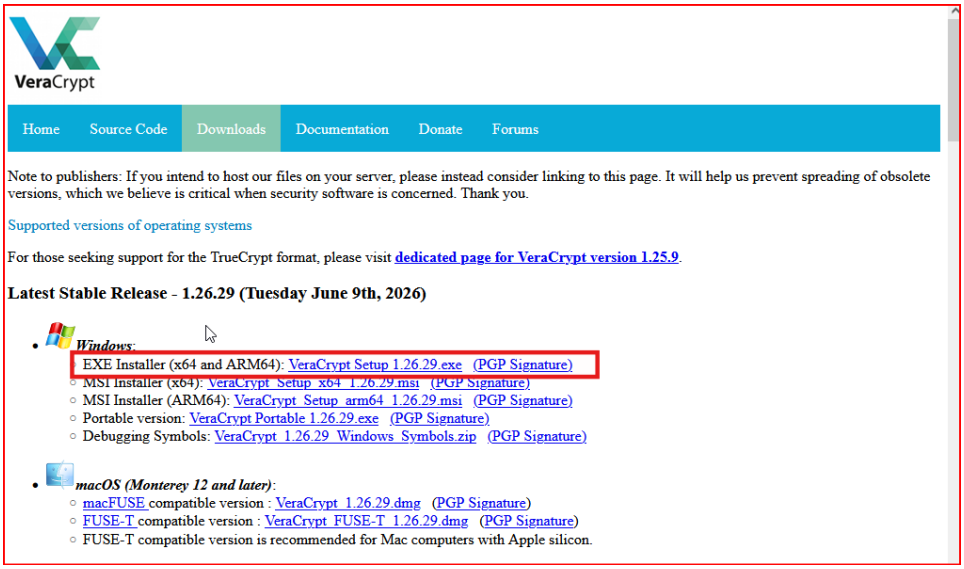
```
C:\Users\cyber>ping 8.8.8.8

Pinging 8.8.8.8 with 32 bytes of data:
Reply from 8.8.8.8: bytes=32 time=35ms TTL=128
Reply from 8.8.8.8: bytes=32 time=34ms TTL=128
Reply from 8.8.8.8: bytes=32 time=35ms TTL=128
Reply from 8.8.8.8: bytes=32 time=34ms TTL=128
```

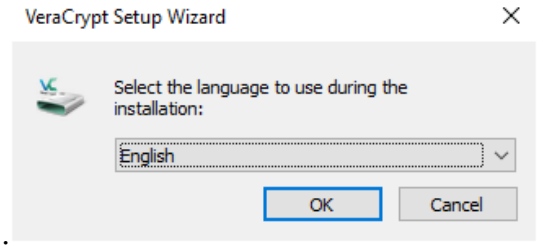
Download and install VeraCrypt.

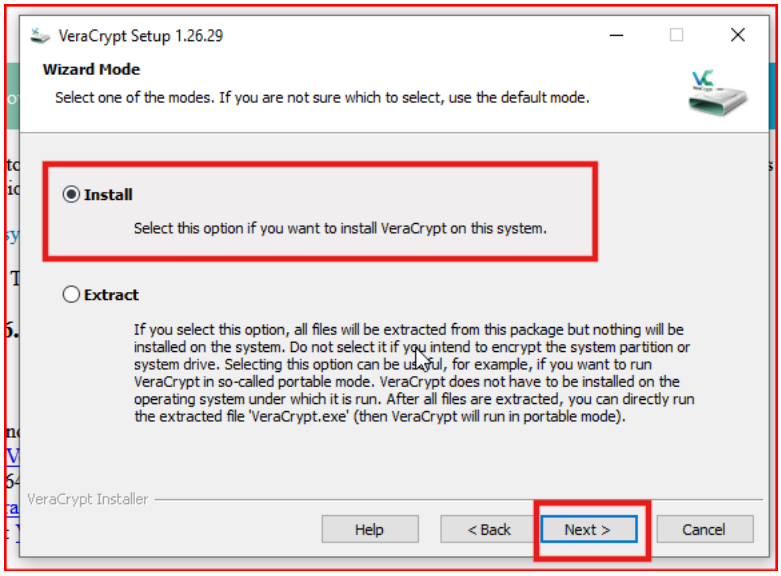
- Click **Downloads** in the top menu (or it should already be on that page).
- Under **Windows**, download the latest installer (exe file).
- Run it once downloaded, and we continue from Step 3 (accept license, Install, etc.).

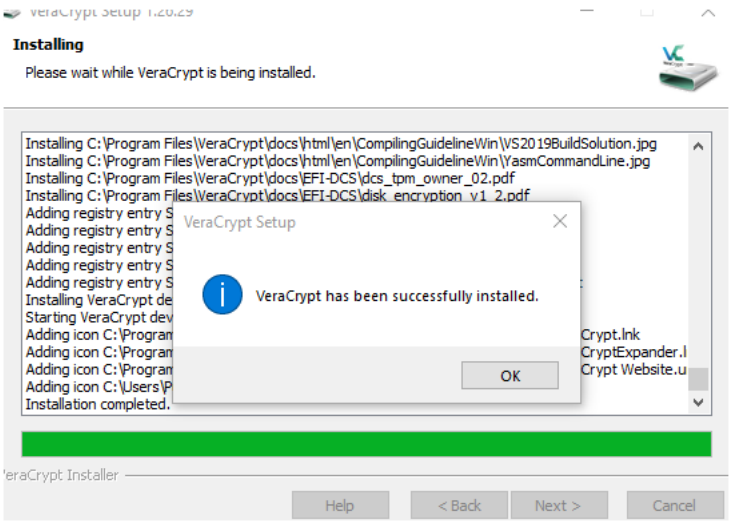




Accept the license agreement, select **Install**, complete setup with default options, then launch VeraCrypt

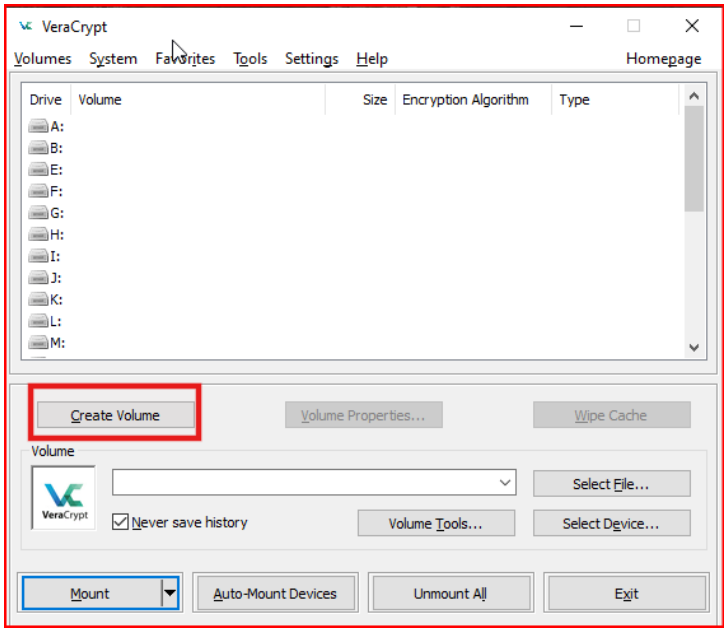




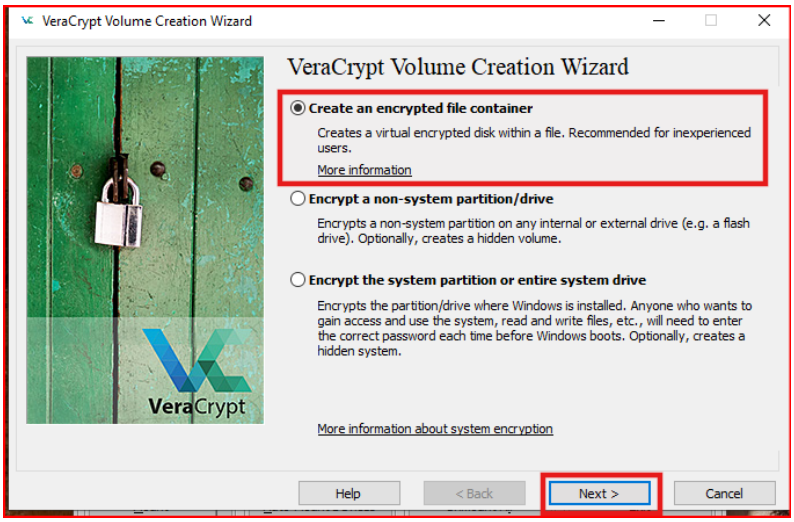


Create Encrypted File Container

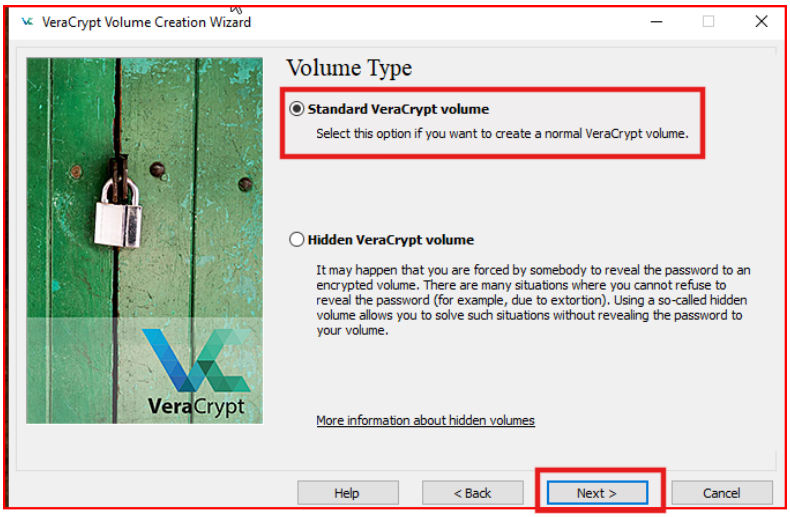
Now click **Create Volume** to start creating the encrypted container.



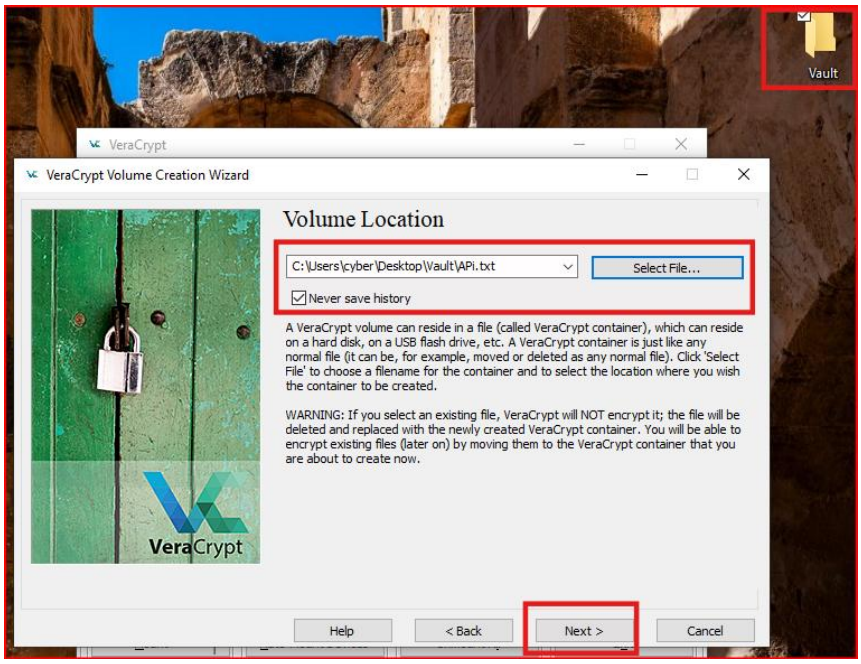
Select **Create an encrypted file container**, click Next.



Select **Standard VeraCrypt volume**, click Next.

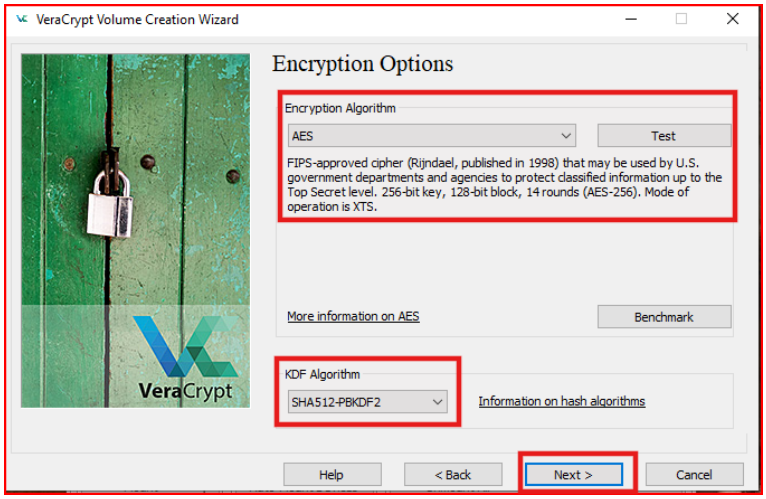


Click **Select File**, choose a location (e.g., Desktop), name it MyVault, click Next.

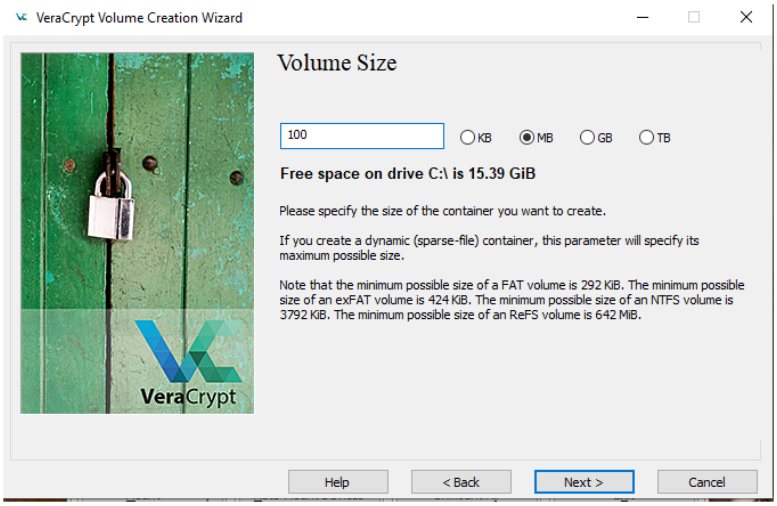


Set Encryption Options (AES) & Volume Size

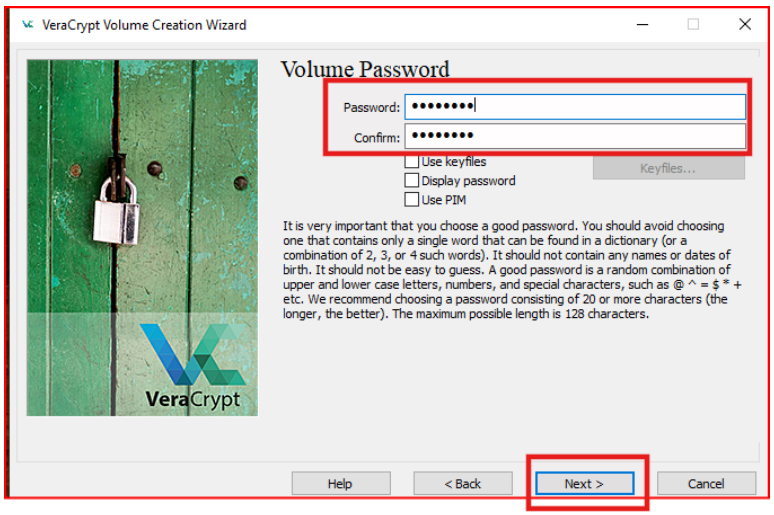
Leave encryption algorithm as **AES**, click Next.



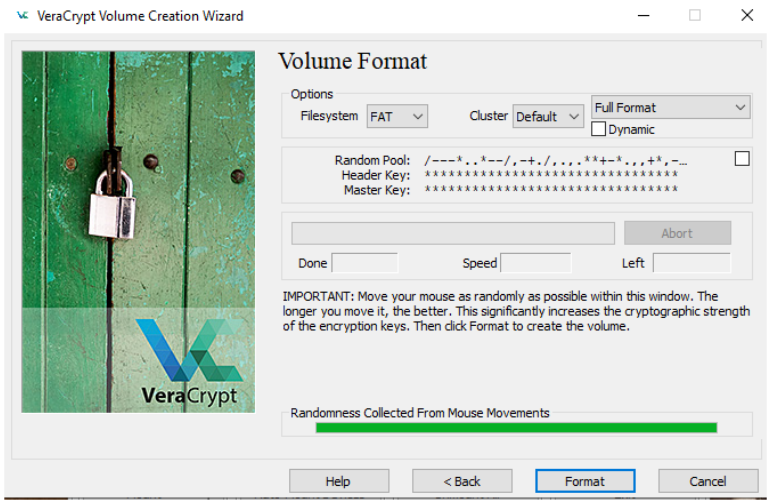
Enter volume size (e.g., 100 MB), click Next.

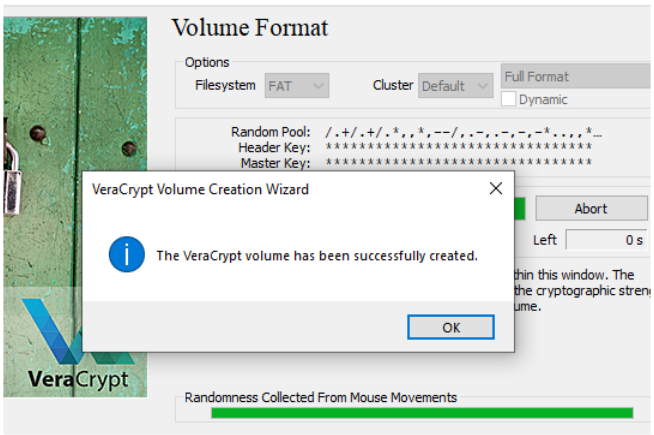


Enter and confirm a password, click Next.



Move the mouse randomly inside the window, then click **Format**. Waiting for completion, click Exit.

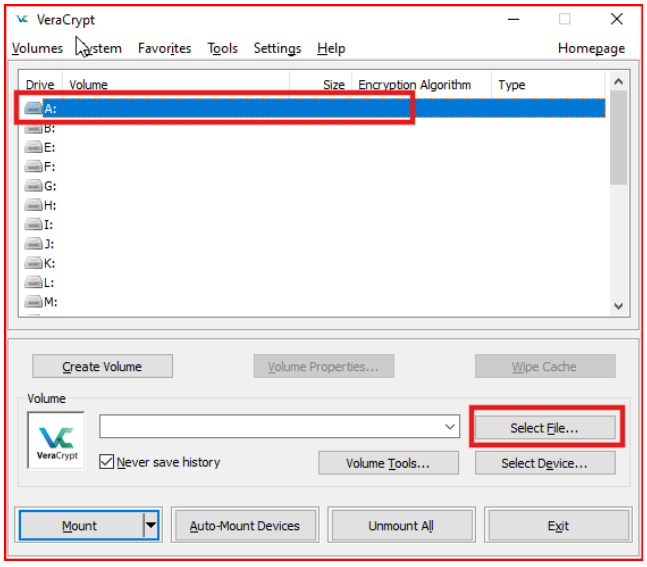


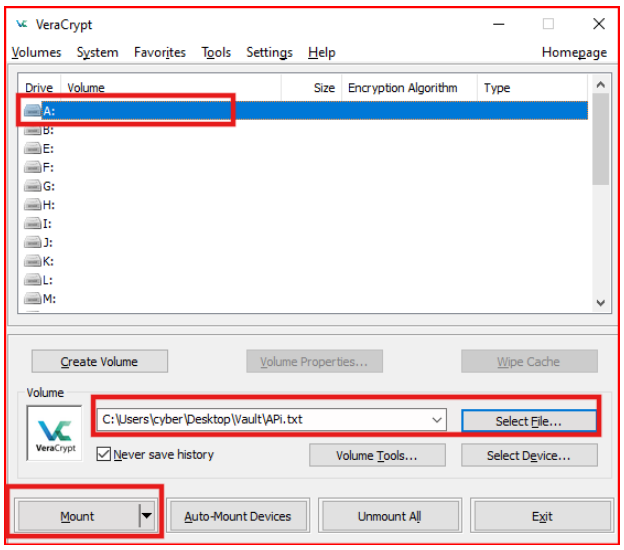


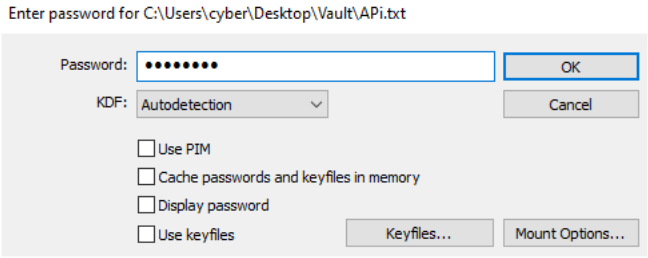


Mount the Encrypted Volume

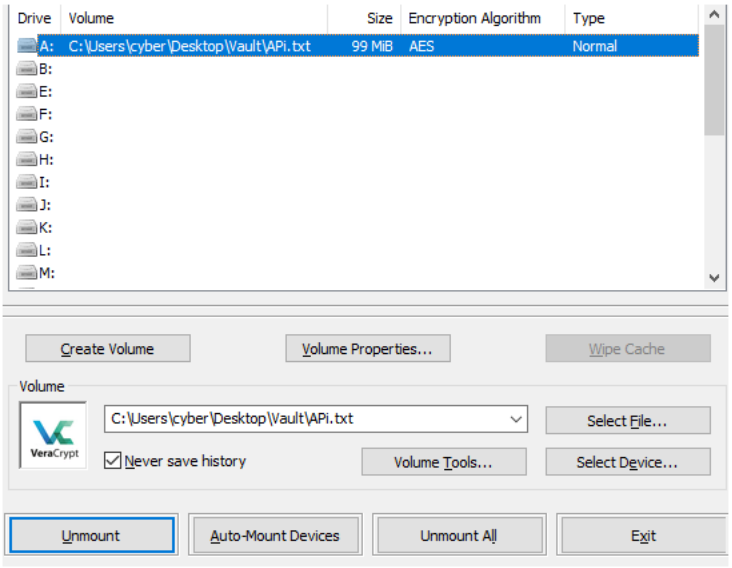
In the main VeraCrypt window, select a drive letter, click **Select File**, choose MyVault, click **Mount**, enter the password.



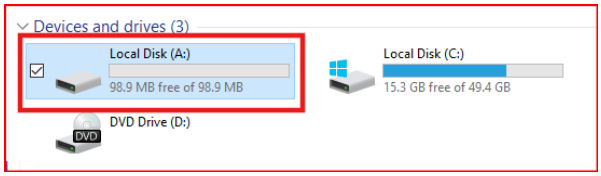




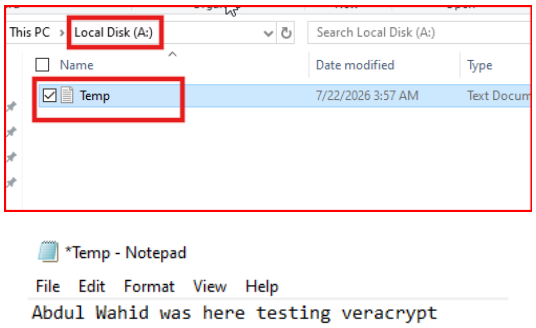
Open **This PC**, confirm the new drive appears, create a test file inside it.



Open **This PC**, double-click drive **A:**

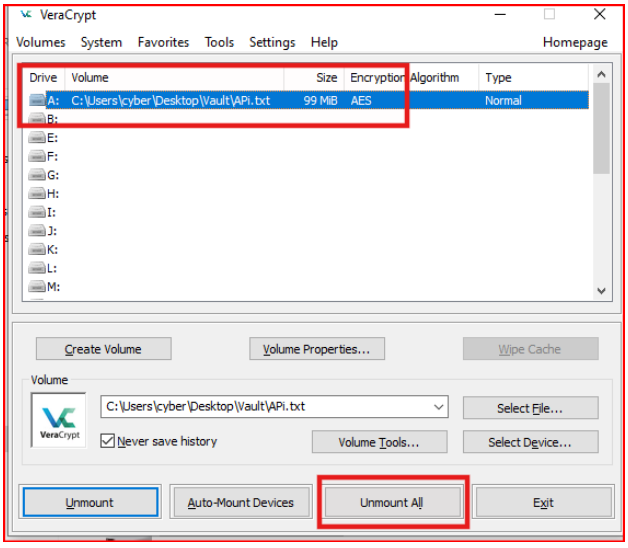


Create a test file inside it (e.g., right-click → New → Text Document, type some text, save).

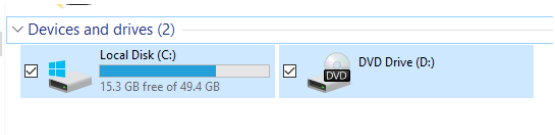


Unmount the Volume

Go back to VeraCrypt, select drive **A:** click **Unmount**.

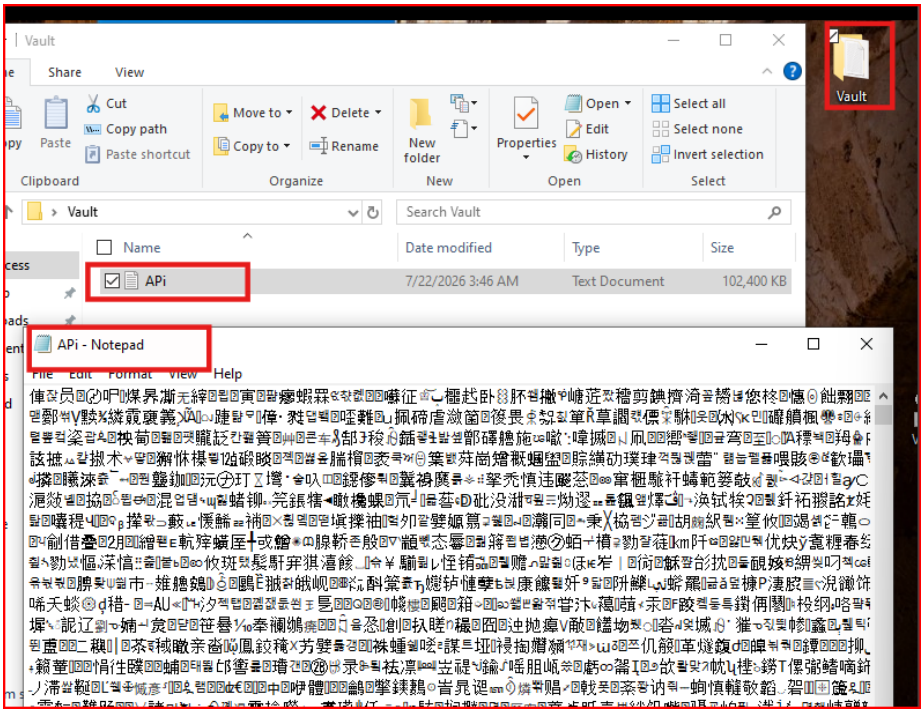


Open **This PC** again — drive **A:** should be gone.



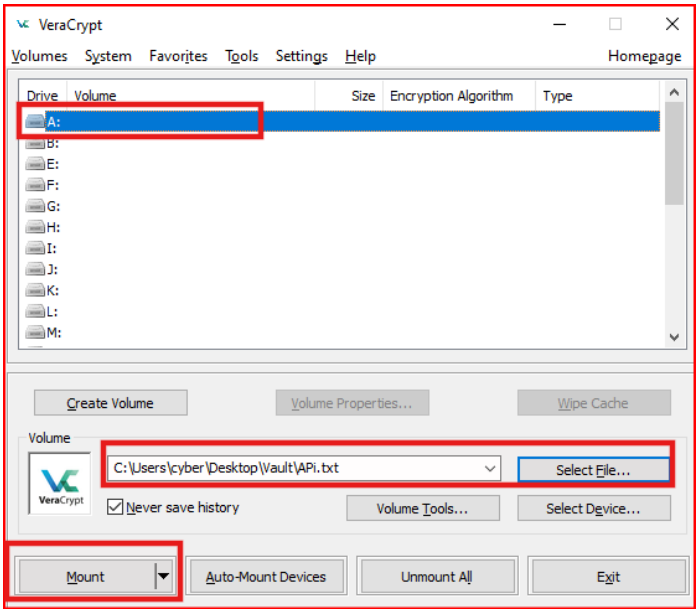
Verify Encryption (Container becomes unreadable)

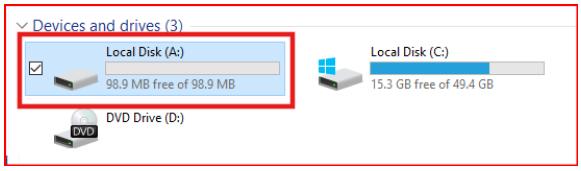
Now check your Desktop — you'll see only APi.txt. If you try to open APi.txt directly (double-click it), it won't open as a text file — it's meaningless encrypted data without VeraCrypt.

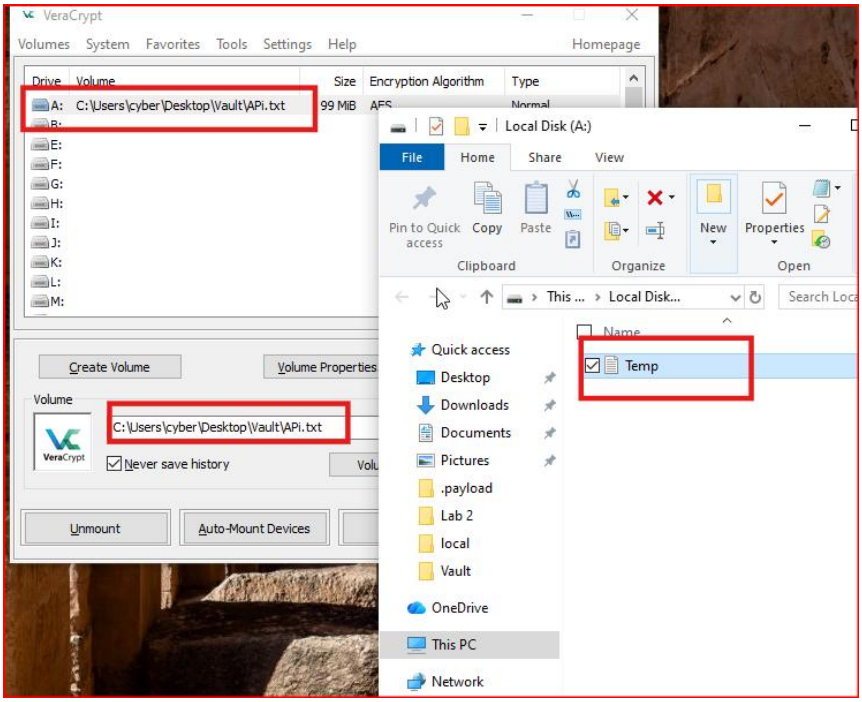


Remount & Confirm Data Integrity

Mount it again with the password, open drive A: and confirm your test file is still there, untouched.







What's happening:

APi.txt on your Desktop isn't really a text file — it's a **container file** (a big blob of encrypted data, 99 MB). VeraCrypt treats it as a "locked box."

When you click **Mount**, VeraCrypt reads that container file and makes Windows treat it as a **real drive** (in your case, drive **A:**) — so you can open it in File Explorer, put files in it, delete files, etc., just like any normal disk.

Anything you save inside drive **A:** actually, gets written **inside that same APi.txt file**, but encrypted. Windows just shows it to you as a normal drive while it's mounted.

When you click **Unmount**, drive **A** disappears completely from File Explorer. The data doesn't vanish — it's still sitting inside APi.txt on your Desktop, just locked/encrypted. You can't read or access it until you mount it again with the correct password

Implementing Transparent Data Encryption (TDE) on SQL Server

Machine: Webserver (Windows Server 2019, SQL Server)

Summary: TDE encrypts an entire SQL Server database "at rest" meaning the actual database files and backups are encrypted on disk, so if someone steals the files, they can't read them without the encryption key.

Objective: Enable TDE on a SQL Server database and verify the database is encrypted.

Key Result: The database is now fully encrypted at rest. Even if someone steals the database files, they cannot read the data without the encryption key. Encryption status was successfully verified using system views.

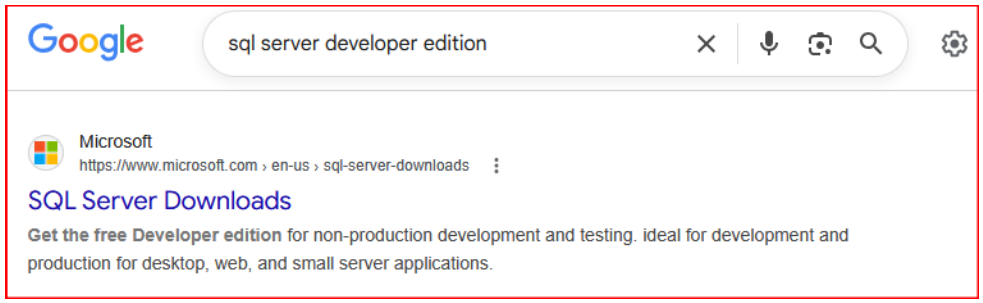


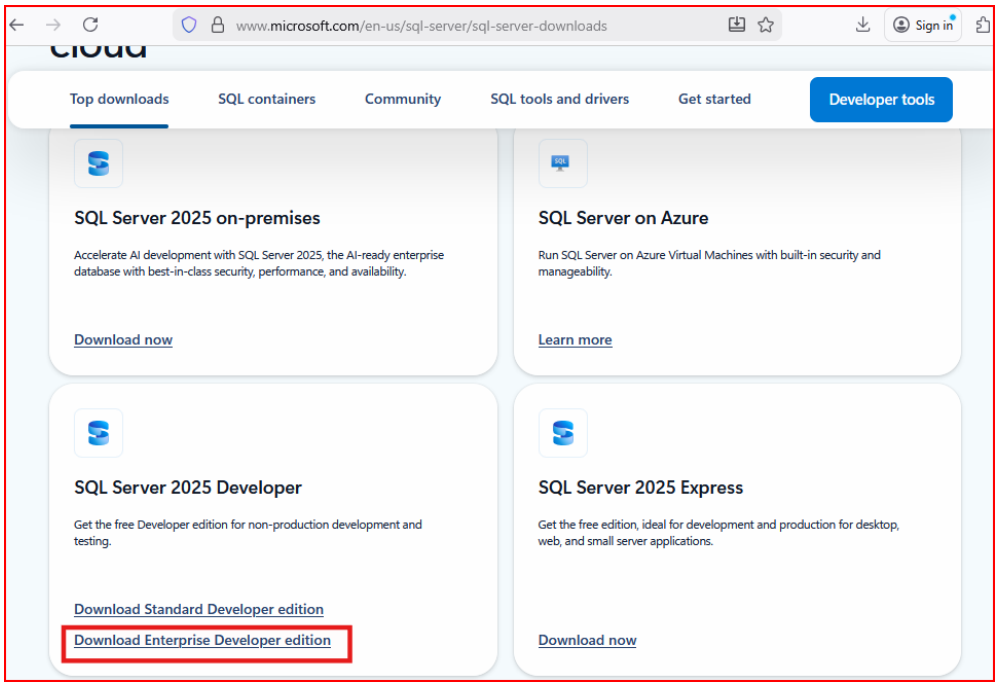
Power on Web Server VM



Download & Install SQL Server Developer Edition

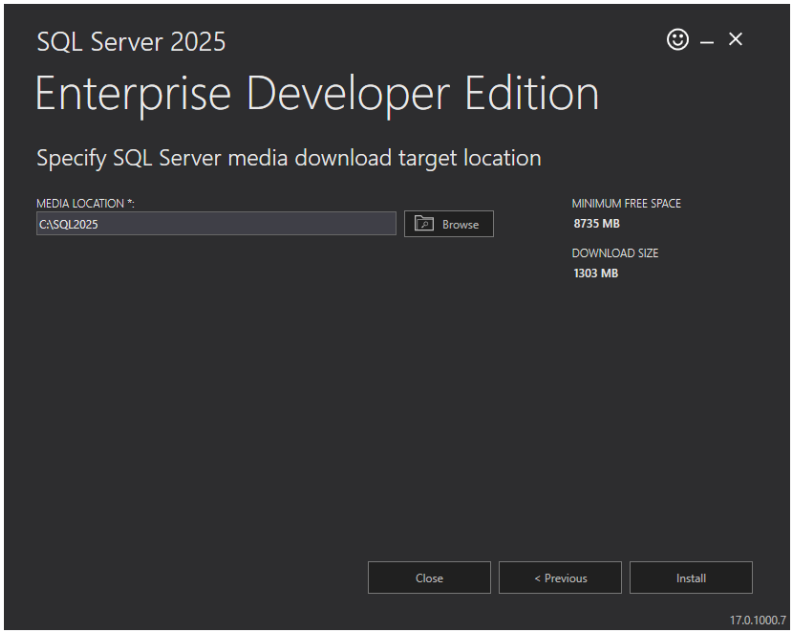
Go to the official Microsoft SQL Server downloads page (microsoft.com) and download **Developer Edition** (Enterprise Developer or Standard Developer — both support TDE).





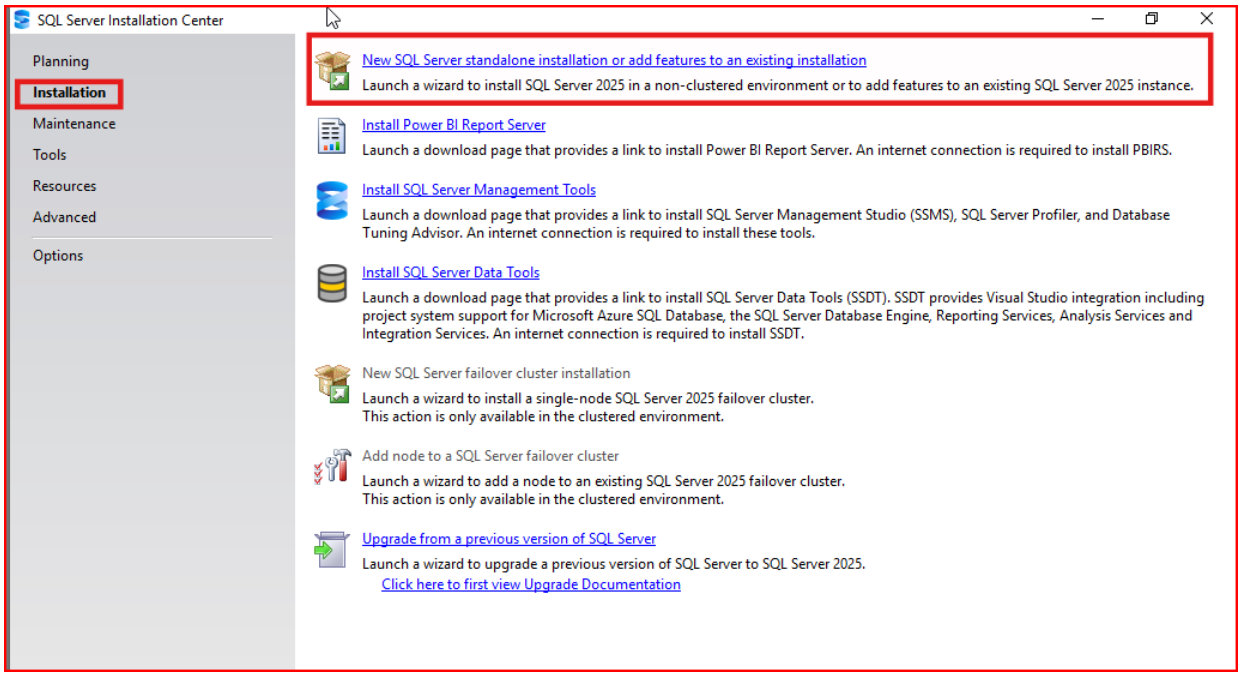
- Run the installer.
- Choose **Basic** or **Custom** installation (Custom gives more control – recommended).

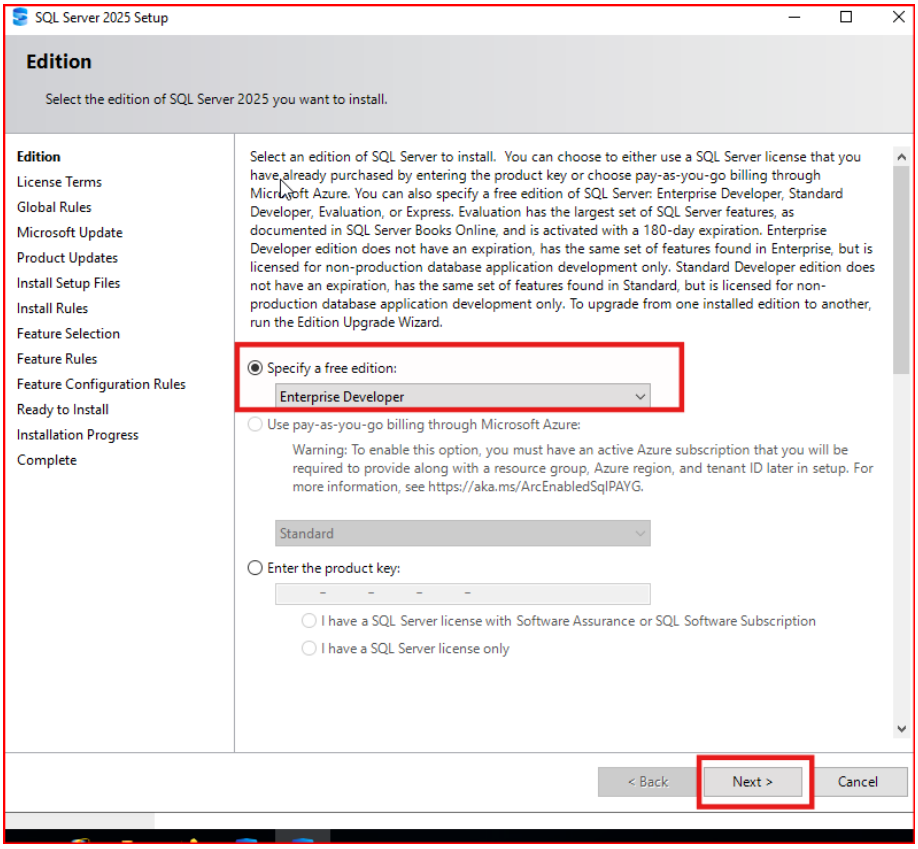




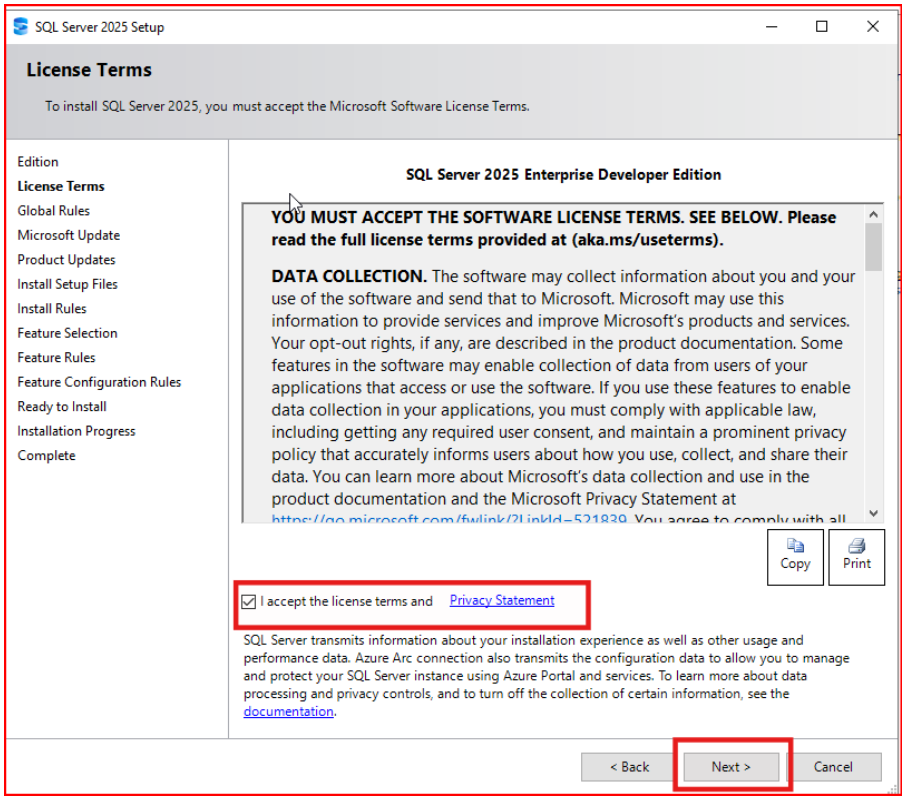
Click **Installation** in the left-hand menu (second item, under "Planning").

Then select **"New SQL Server stand-alone installation or add features to an existing installation"** from the list that appears.



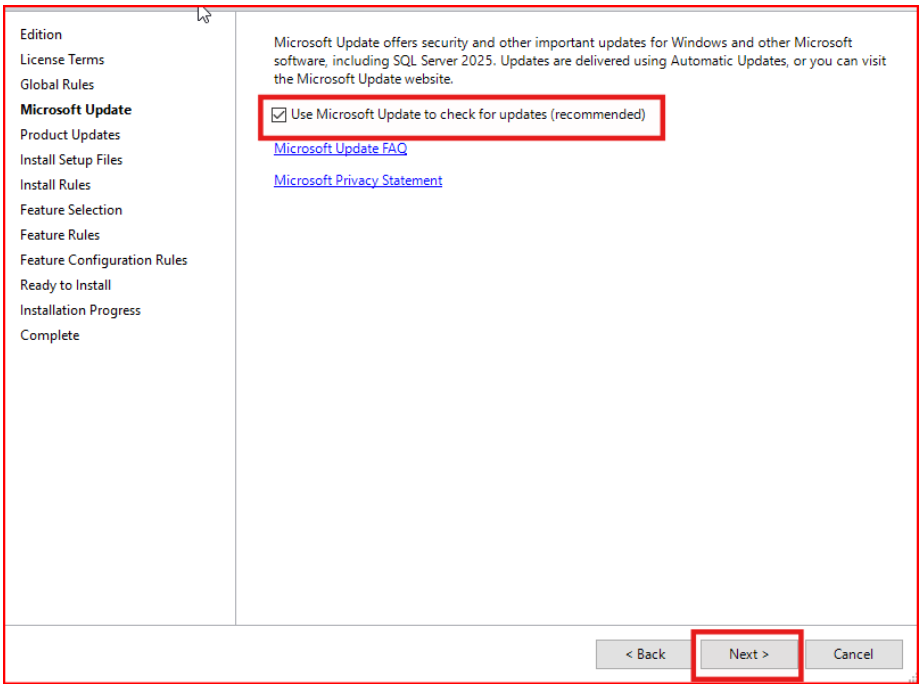


Accept license terms → Next

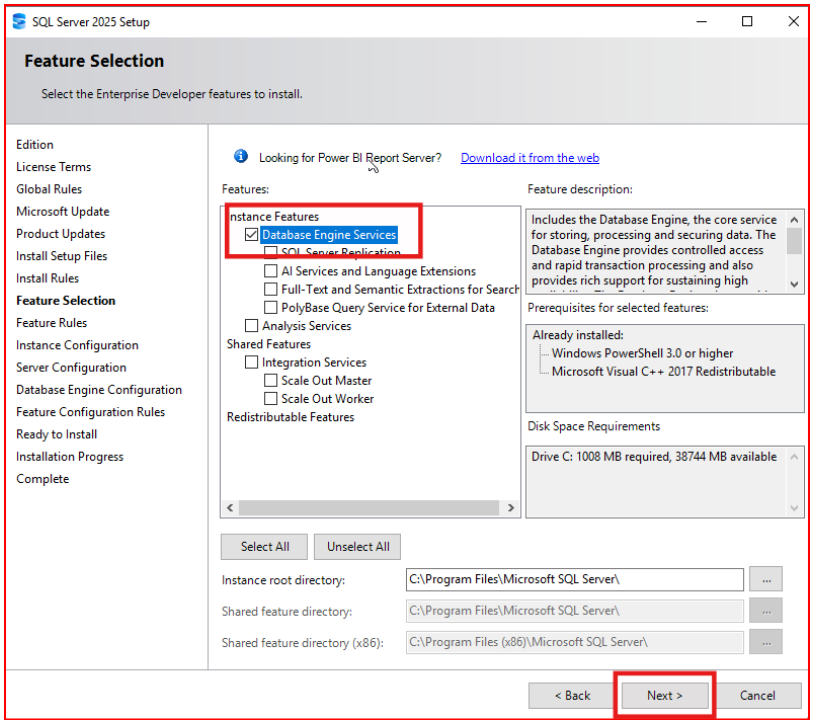


Let it check for updates → Next

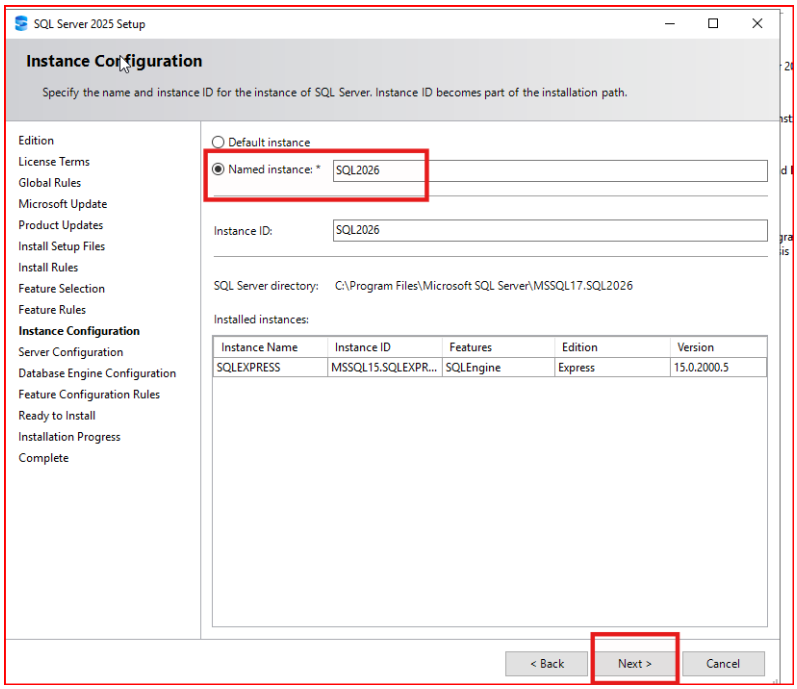
Install setup files → Next



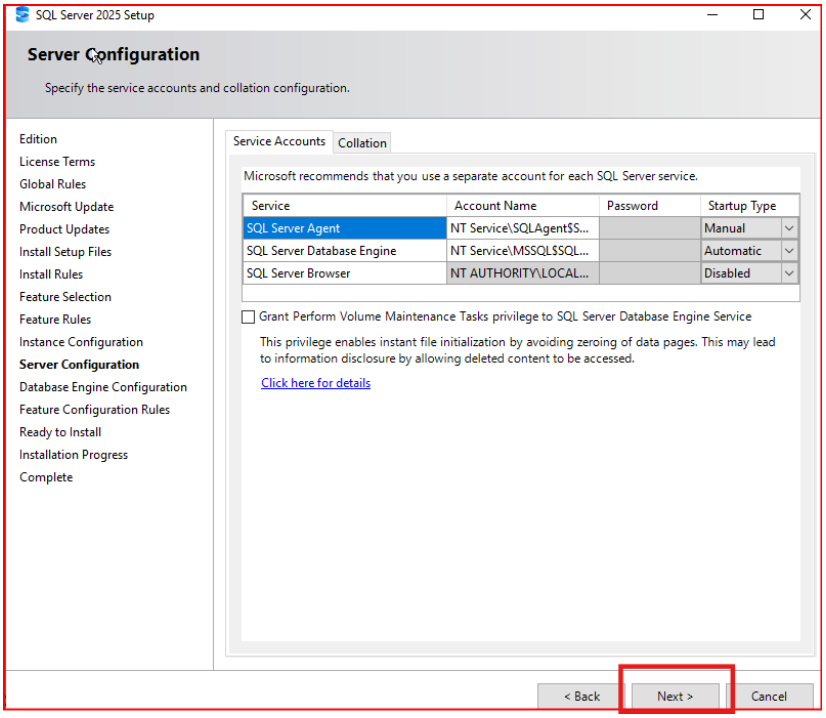
On **Feature Selection**: check **Database Engine Services** → Next



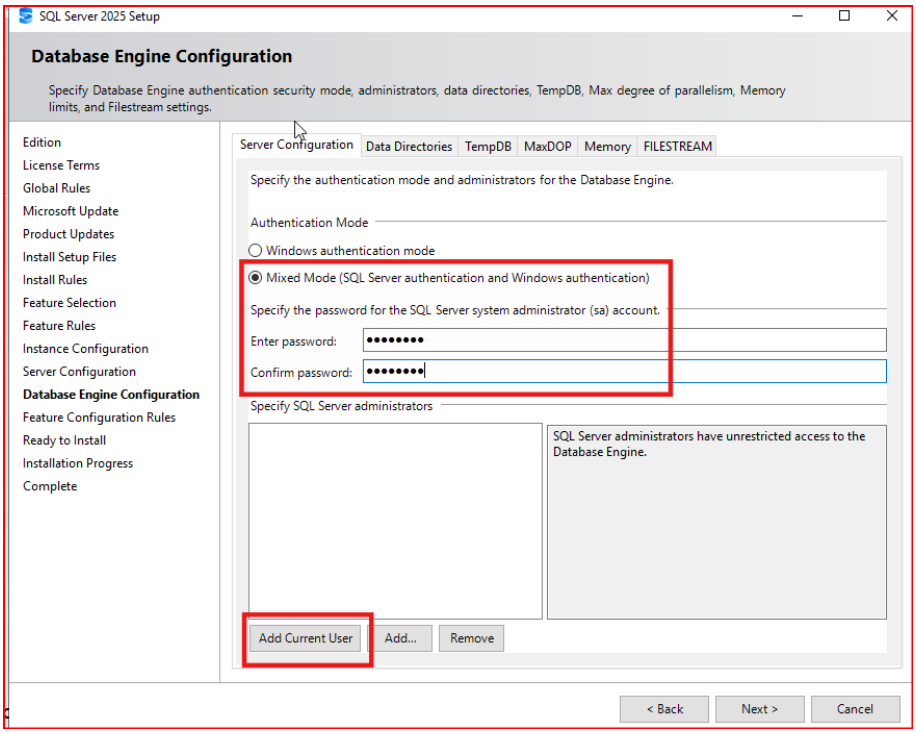
Instance **Configuration**: choose **Named instance**, give it a simple name (e.g., SQL2025) this way it installs alongside your old instance without needing to uninstall it first, avoiding conflicts

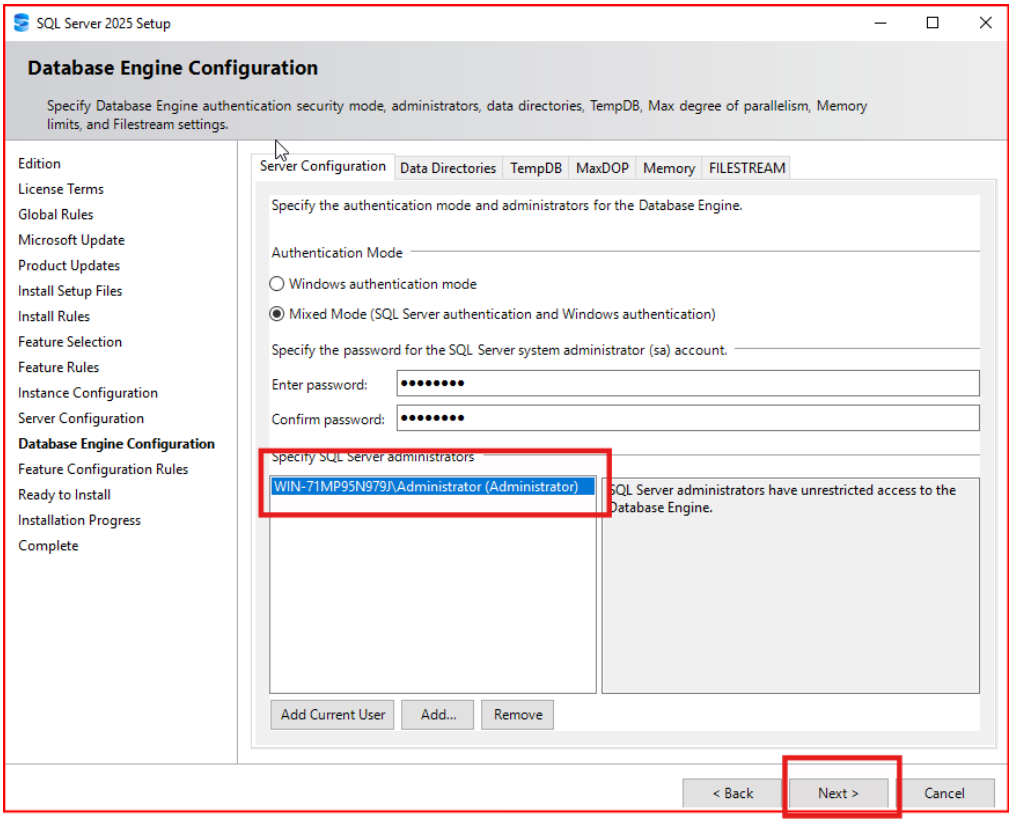


On **Server Configuration** → Next (defaults are fine)

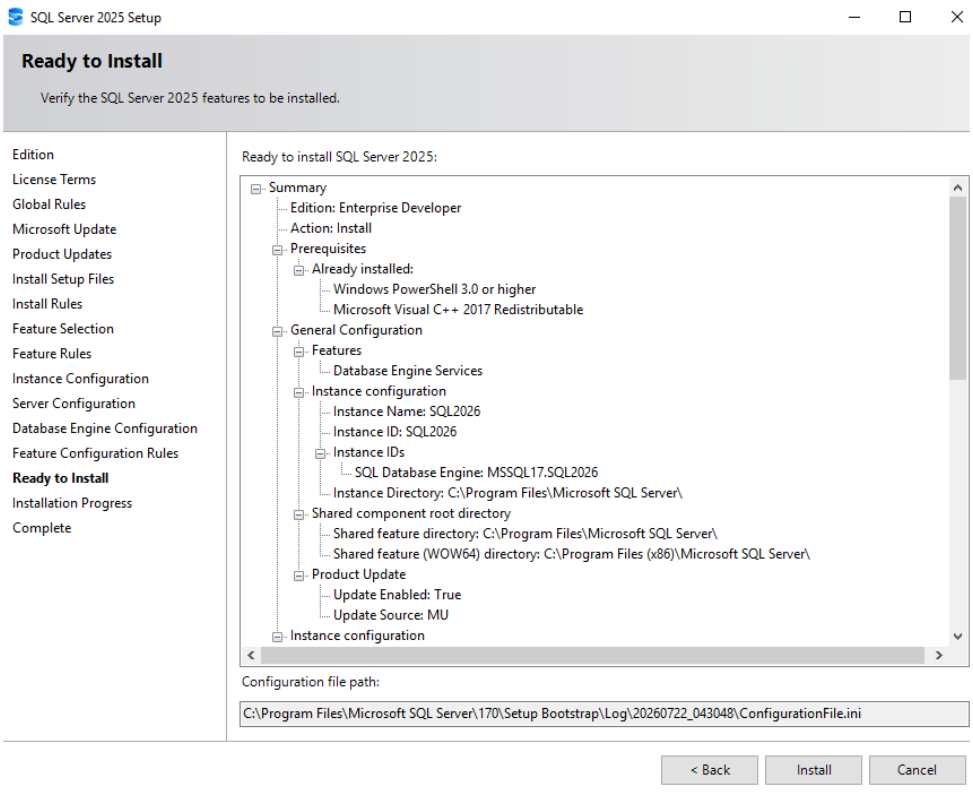


On **Database Engine Configuration** → **Server Configuration** tab: select **Mixed Mode**, set sa password to qwerty@123, click **Add Current User**

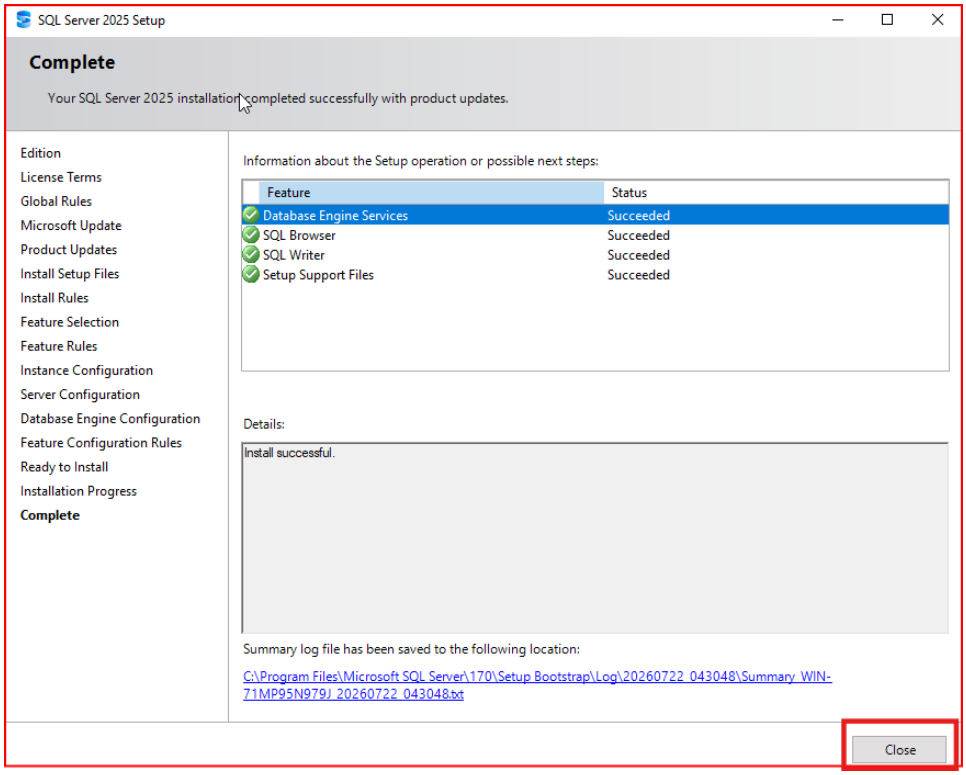




Click **Next** through remaining screens → **Install**



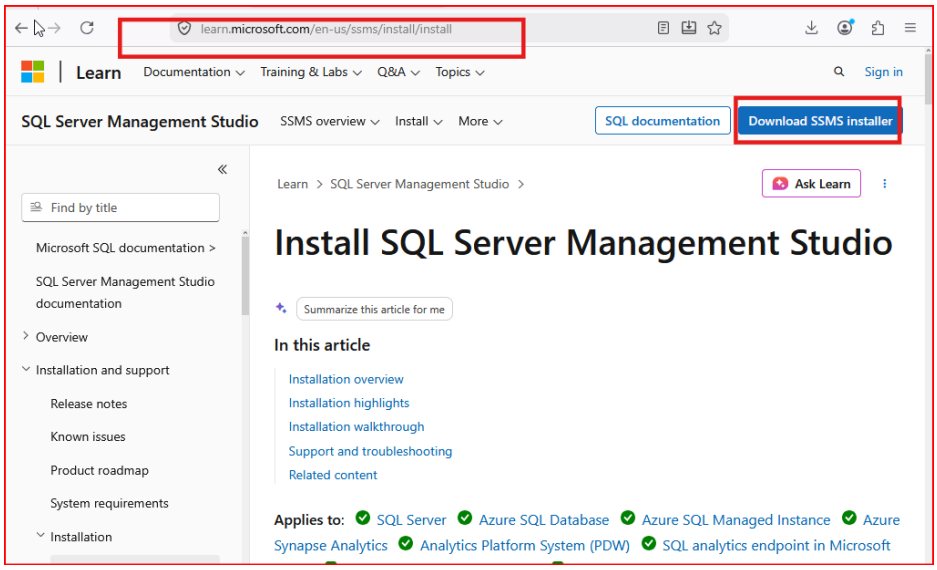
SQL Server 2025 Developer Edition installed successfully. Click **Close**.

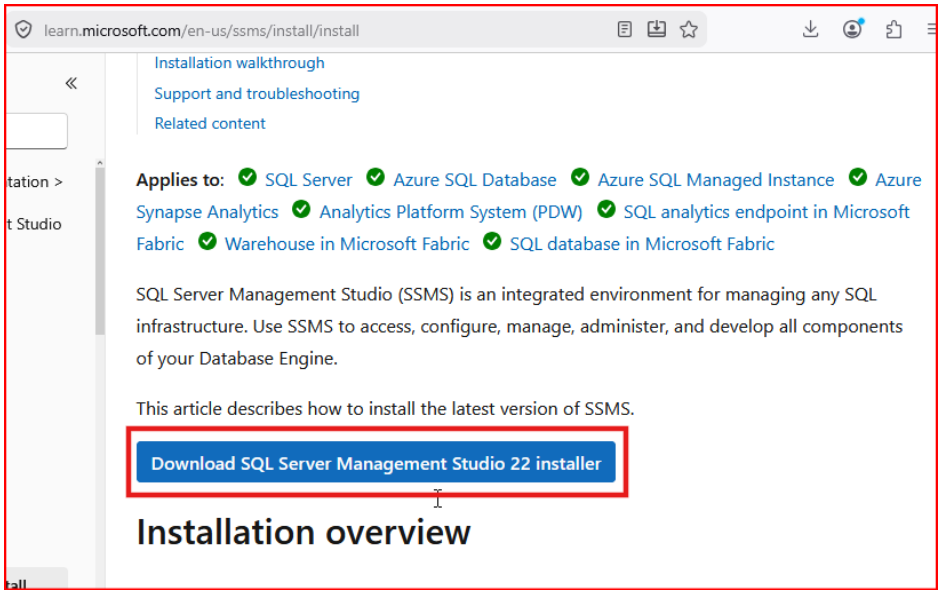


Install SQL Server Management Studio (SSMS)

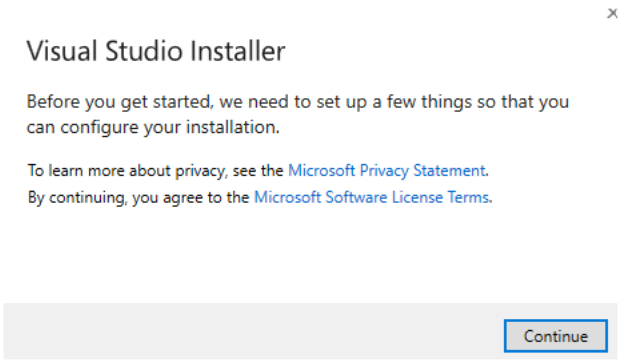
Search Google for **"download SQL Server Management Studio"** and go to the official Microsoft page (learn.microsoft.com/ssms).

Download the SSMS installer (SSMS-Setup-ENU.exe).

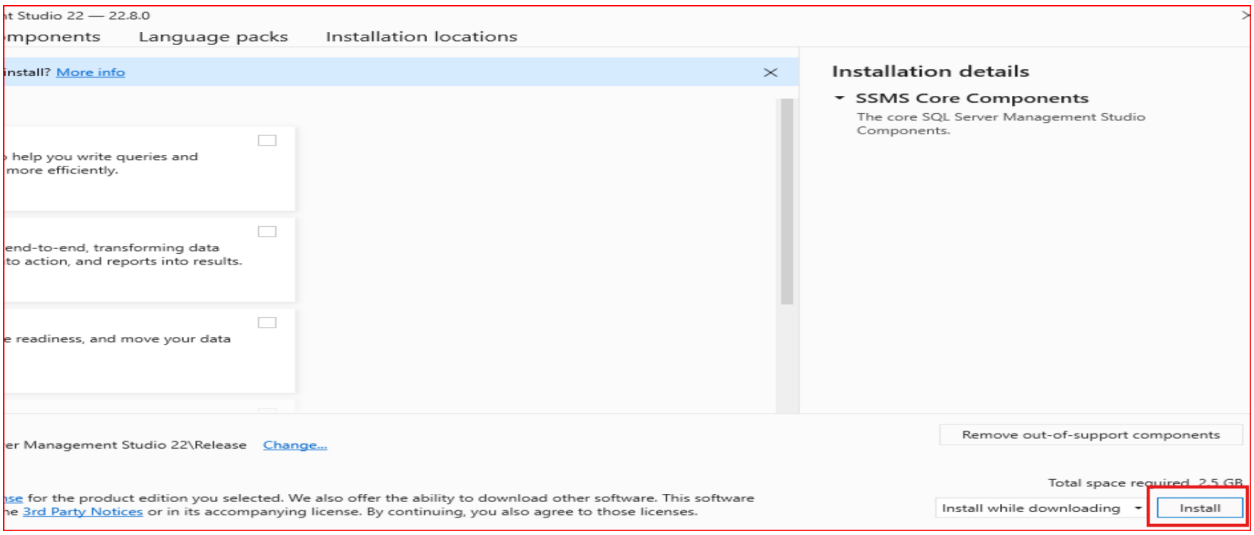


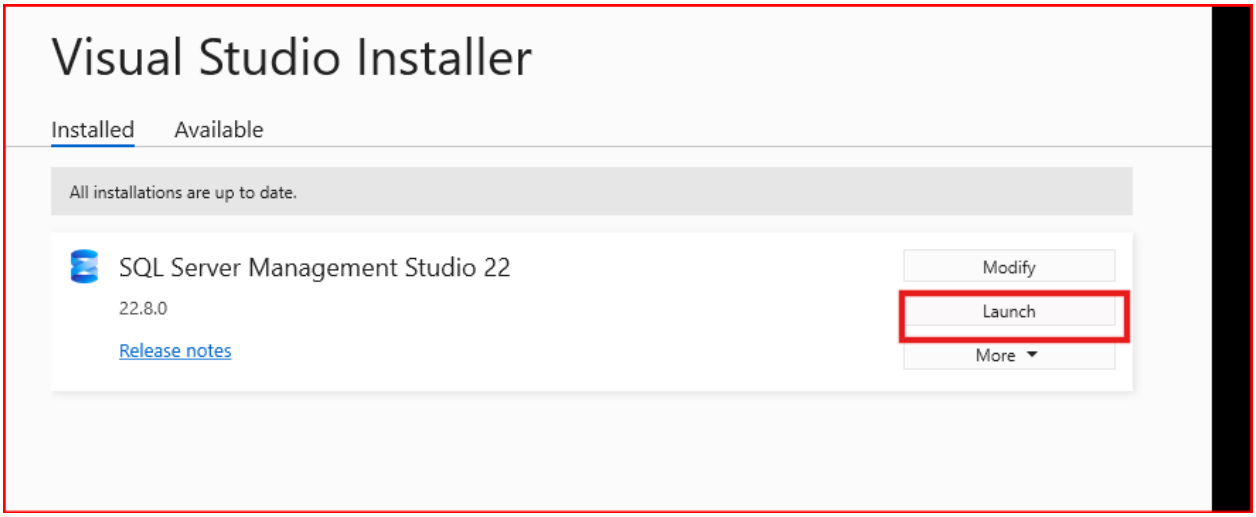


Run it → click **Install** → wait for it to complete → click **Close** (may require a restart).



Then search Start menu for **"SQL Server Management Studio"** and open it.

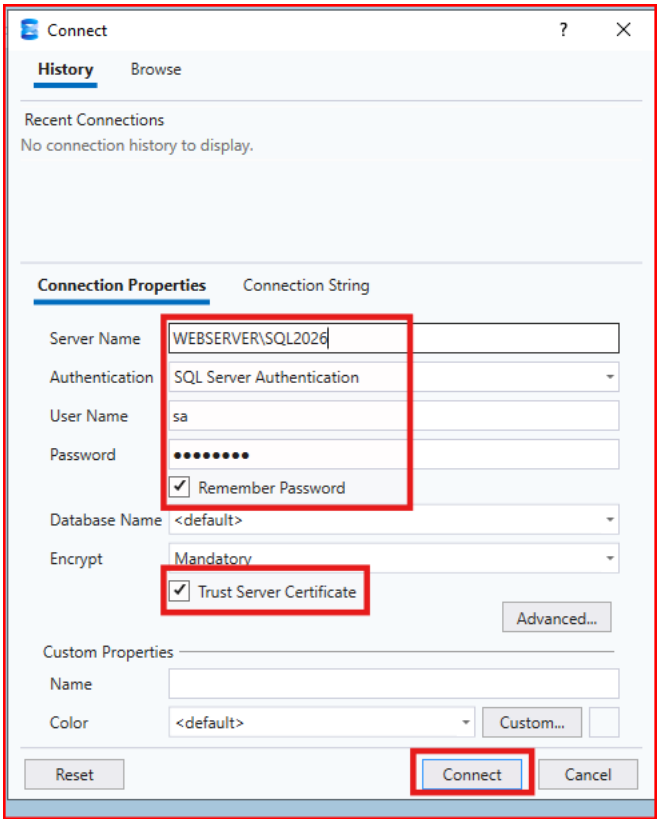




Add Server Name – SQL Server Authentication

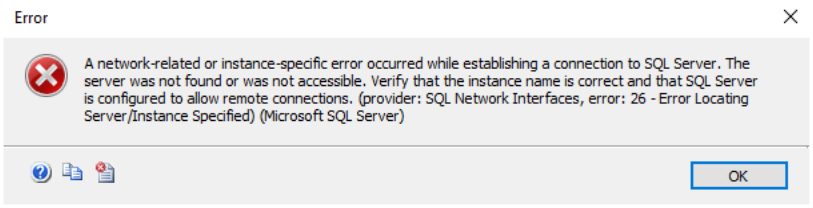
Check Trust Server Certificates

Click Connect

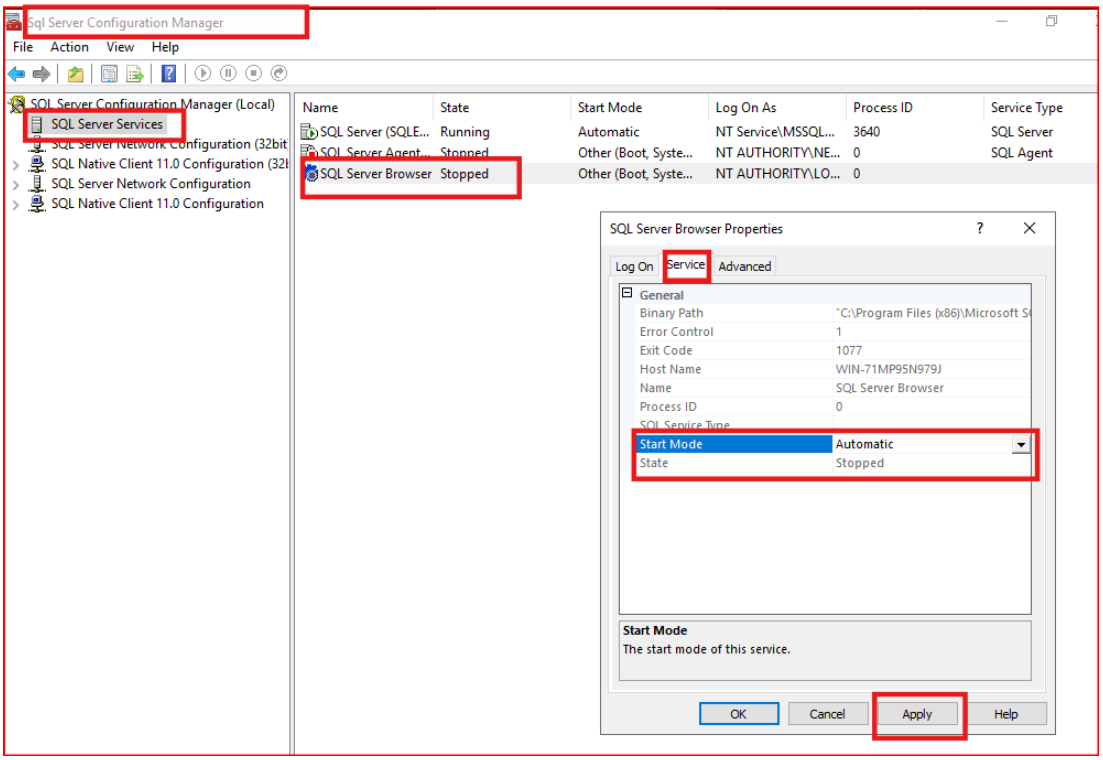


Error Handling (Optional)

If you find this error



- Open **SQL Server 2025 Configuration Manager** (search in Start menu).
- Click **SQL Server Services** in the left panel.
- Look for **SQL Server Browser** in the list on the right.
- Right-click it → **Start** (if it says "Stopped"). Also, right-click → **Properties** → **Service** tab → set **Start Mode** to **Automatic,** so it stays on.

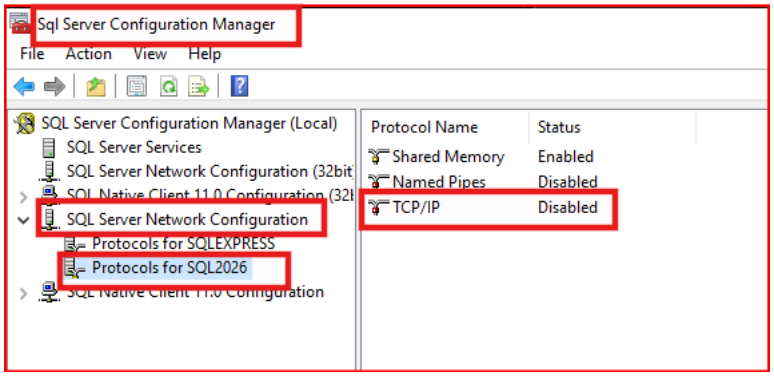


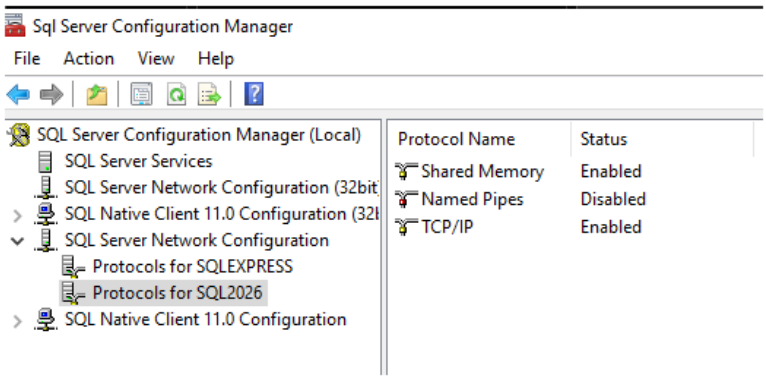
Also enable TCP/IP (common second cause):

In the same Configuration Manager, expand **SQL Server Network Configuration** → click **Protocols for SQL2026**.

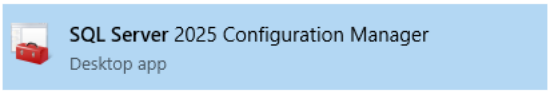
Make sure **TCP/IP** is set to **Enabled** (right-click it → Enable if it isn't).

If you just enabled it, restart the **SQL Server (SQL2026)** service (right-click → Restart) for it to take effect.

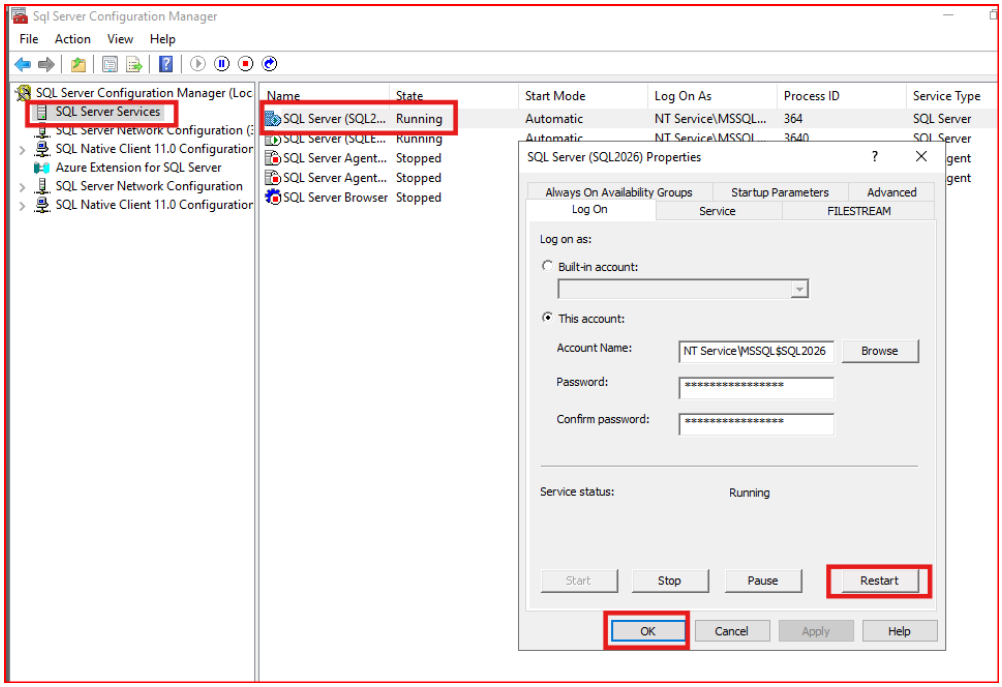




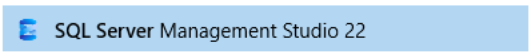
Open SQL Server 2025 Configuration manager



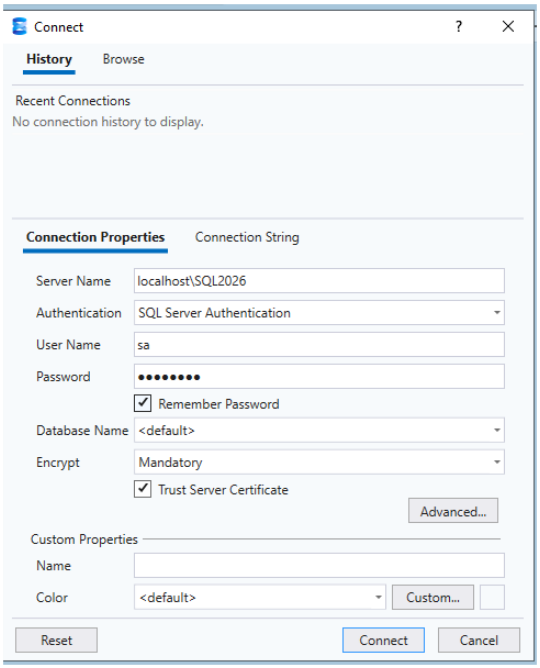
check **SQL Server (SQL2026)** confirm it says **Running**. If not, right-click → Start.



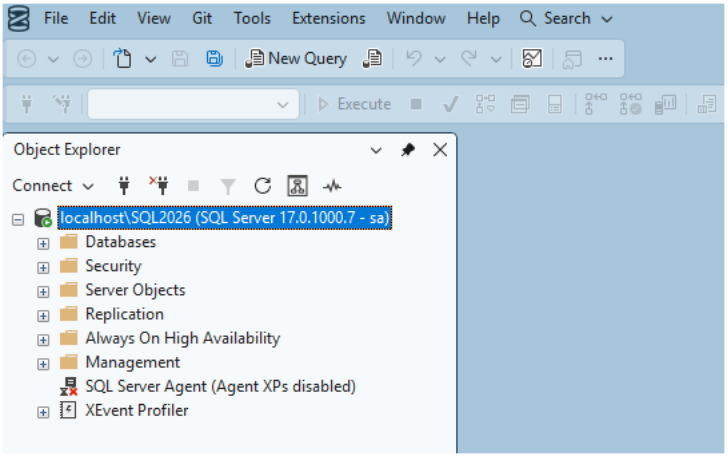
Now opens again SSMS



Add the following details and click on connect



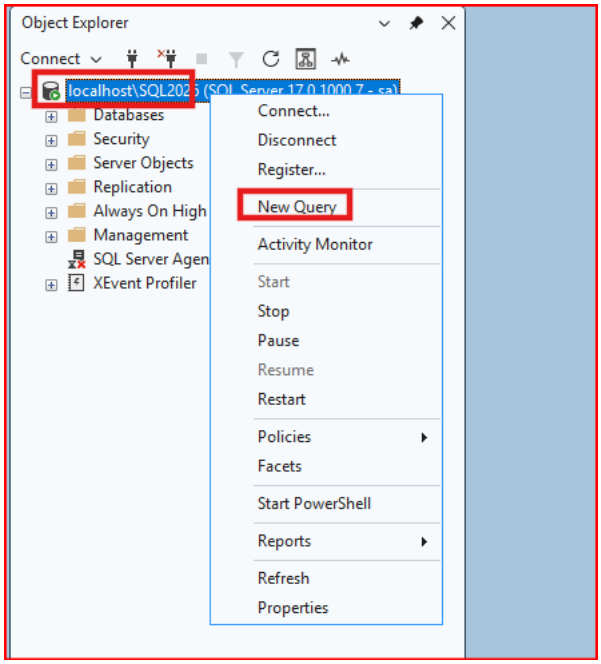
You're now connected to localhost\SQL2026 with sa.

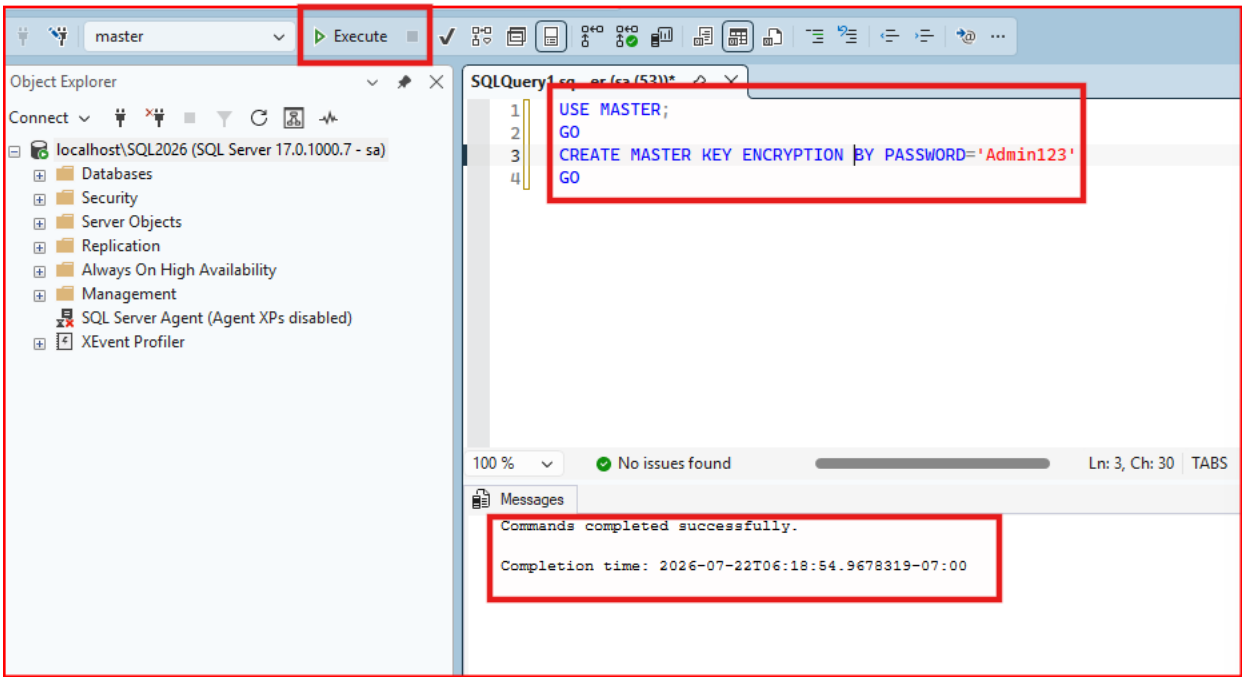


Create Database Master Key

Click **New Query**.

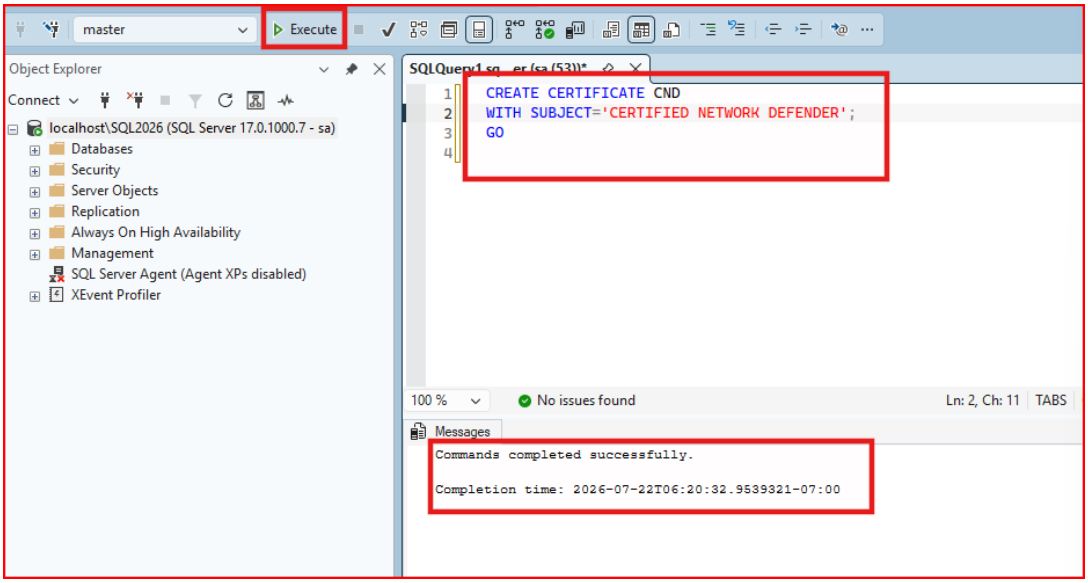
Run this to create the master key:



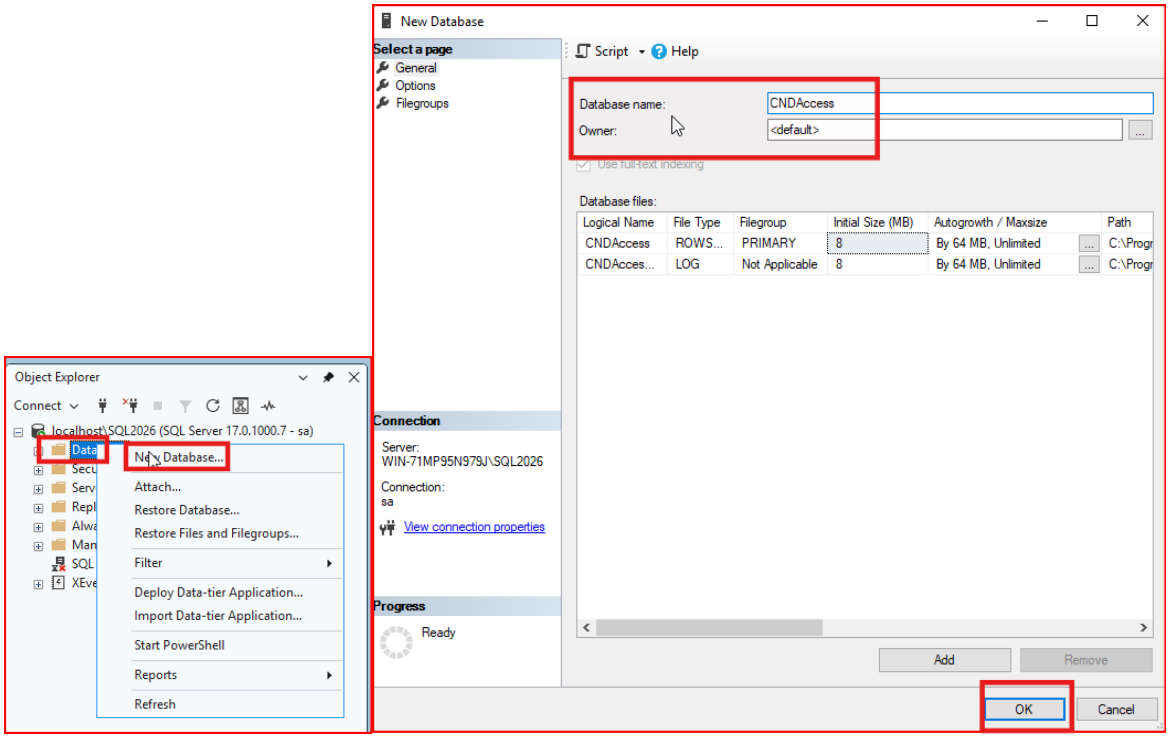


Create Certificate

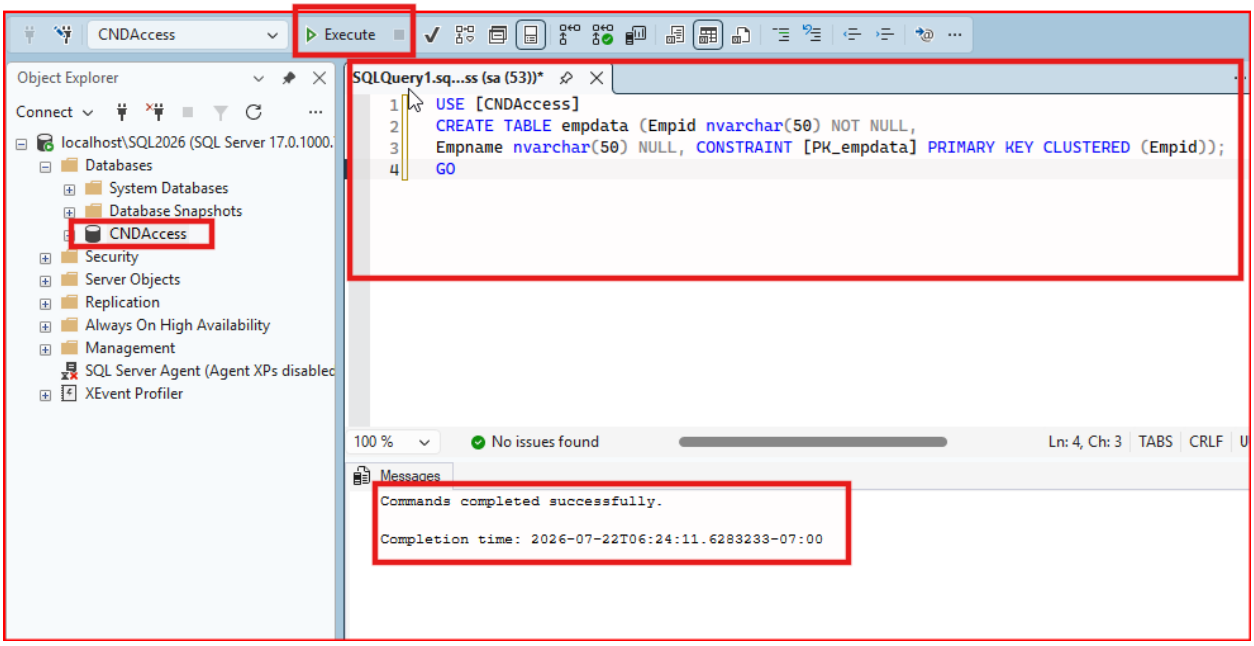
Run this to create the certificate:



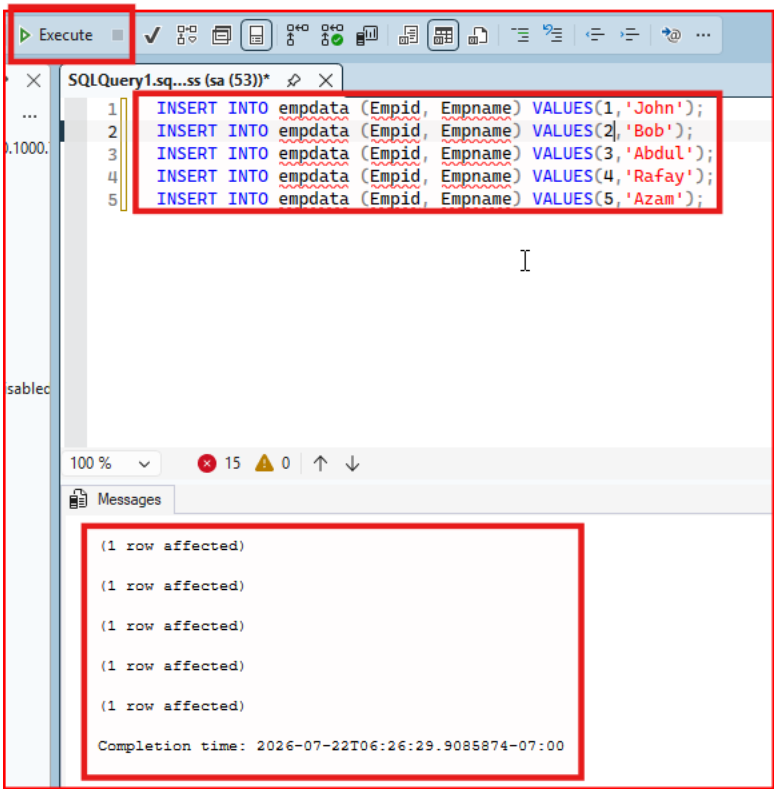
Right-click **Databases** → **New Database** → name it CNDAccess → click OK.



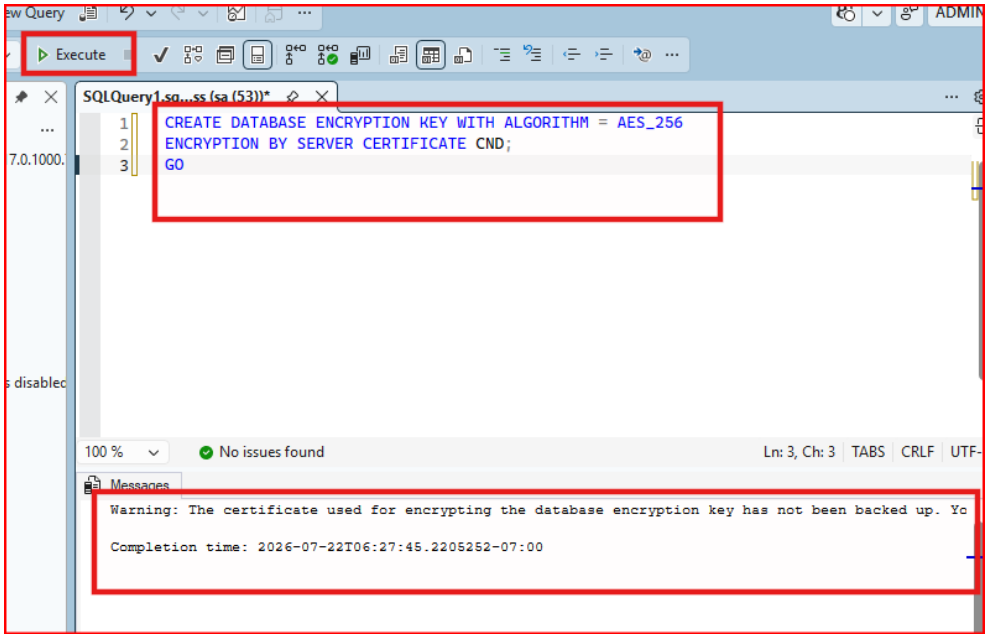
Open a New Query (same sa connection) and run:



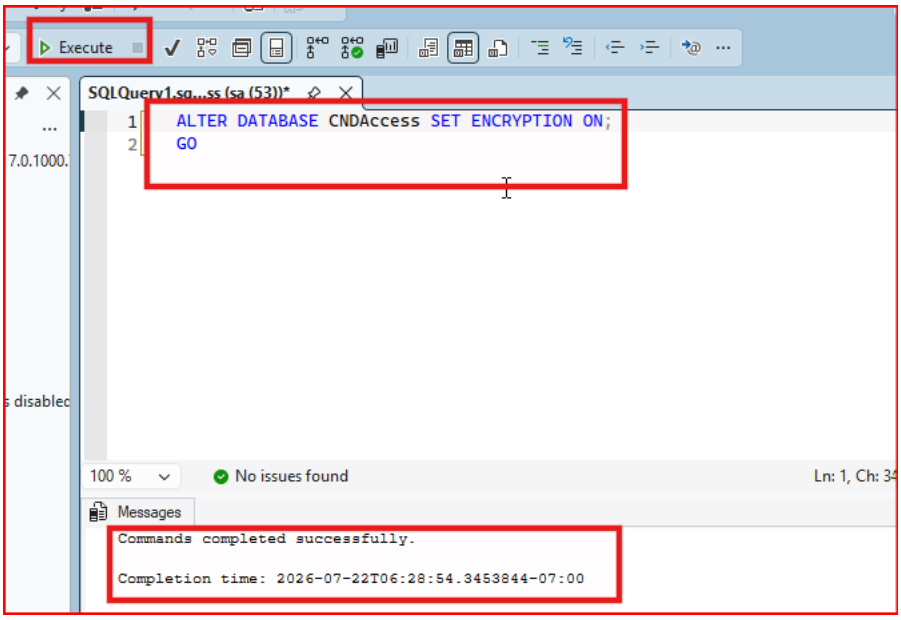
Insert sample records:



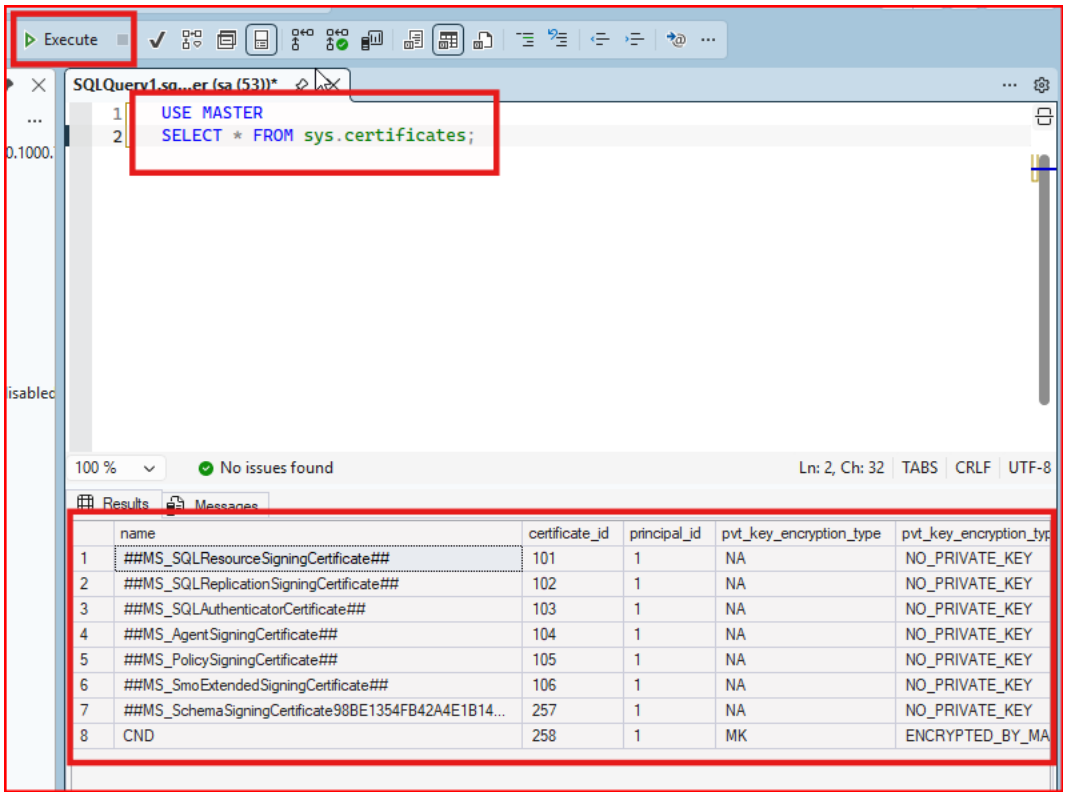
Create Database Encryption Key



Enable Transparent Data Encryption (TDE)

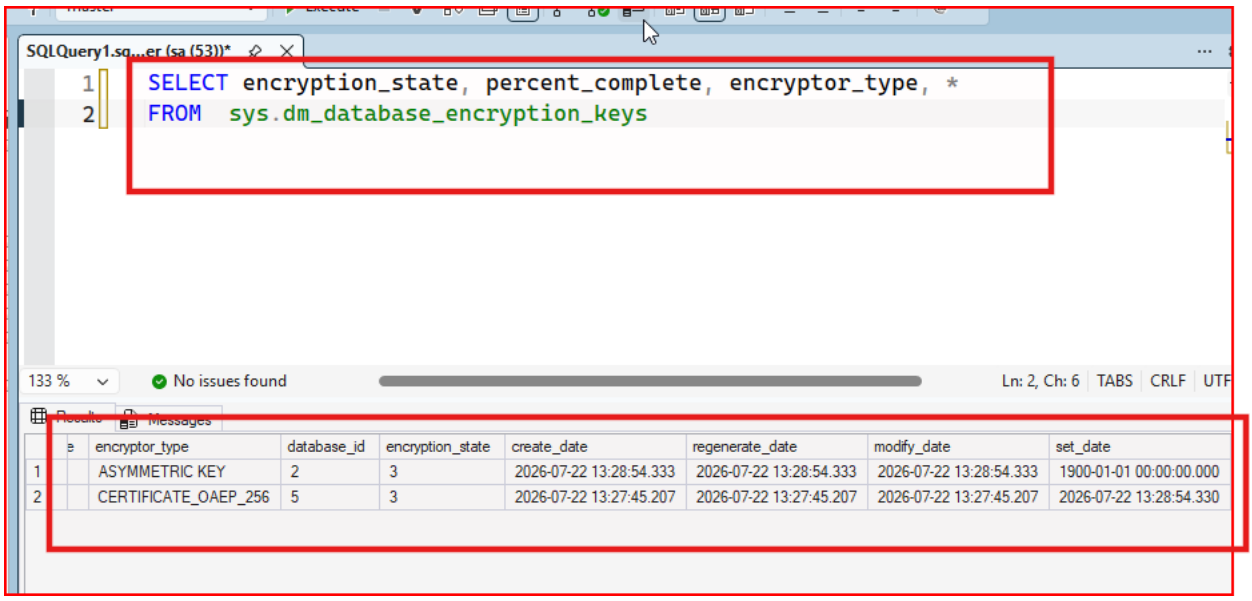


Verify the certificate exists:



	name	certificate_id	principal_id	pvt_key_encryption_type	pvt_key_encryption_typ
1	##MS_SQLResourceSigningCertificate##	101	1	NA	NO_PRIVATE_KEY
2	##MS_SQLReplicationSigningCertificate##	102	1	NA	NO_PRIVATE_KEY
3	##MS_SQLAuthenticatorCertificate##	103	1	NA	NO_PRIVATE_KEY
4	##MS_AgentSigningCertificate##	104	1	NA	NO_PRIVATE_KEY
5	##MS_PolicySigningCertificate##	105	1	NA	NO_PRIVATE_KEY
6	##MS_SmoExtendedSigningCertificate##	106	1	NA	NO_PRIVATE_KEY
7	##MS_SchemaSigningCertificate98BE1354FB42A4E1B14...	257	1	NA	NO_PRIVATE_KEY
8	CND	258	1	MK	ENCRYPTED_BY_MA

Verify the database is encrypted:



	encryptor_type	database_id	encryption_state	create_date	regenerate_date	modify_date	set_date
1	ASYMMETRIC KEY	2	3	2026-07-22 13:28:54.333	2026-07-22 13:28:54.333	2026-07-22 13:28:54.333	1900-01-01 00:00:00.000
2	CERTIFICATE_OAEP_256	5	3	2026-07-22 13:27:45.207	2026-07-22 13:27:45.207	2026-07-22 13:27:45.207	2026-07-22 13:28:54.330

What's Actually Happening?

- In a normal SQL Server database, the data files (.mdf and .ldf) and backups are stored on the disk in plain text. If someone steals these files (for example, by copying them from the server or taking backup), they can easily attach the database on another server and read all the data.
- Transparent Data Encryption (TDE) solves this problem by encrypting the entire database at the file level.
- **Here's how it works:**
    - A Database Master Key and a Certificate are created in the master database.
    - A Database Encryption Key is then created and protected by that certificate.
    - When TDE is enabled on the database, SQL Server automatically encrypts every page of data before writing it to the disk.
    - When the data is read back, SQL Server automatically decrypts it in memory.
- The encryption and decryption happen transparently — the application and users don't need to change anything. They continue to work with the database normally.
- However, the physical database files and backups on the disk are now completely encrypted. Without the correct certificate and keys, no one can open or read the database — even if they steal the files.

Implementing Always Encrypted in SQL Server

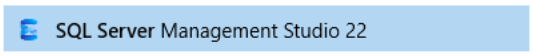
Machine: Web Server (using your SQL2026 instance) Use Server 2

Summary: Always Encrypted encrypts specific columns on the client side — the encryption keys are never revealed to the database engine itself, so even admins with full database access can't read the actual sensitive values.

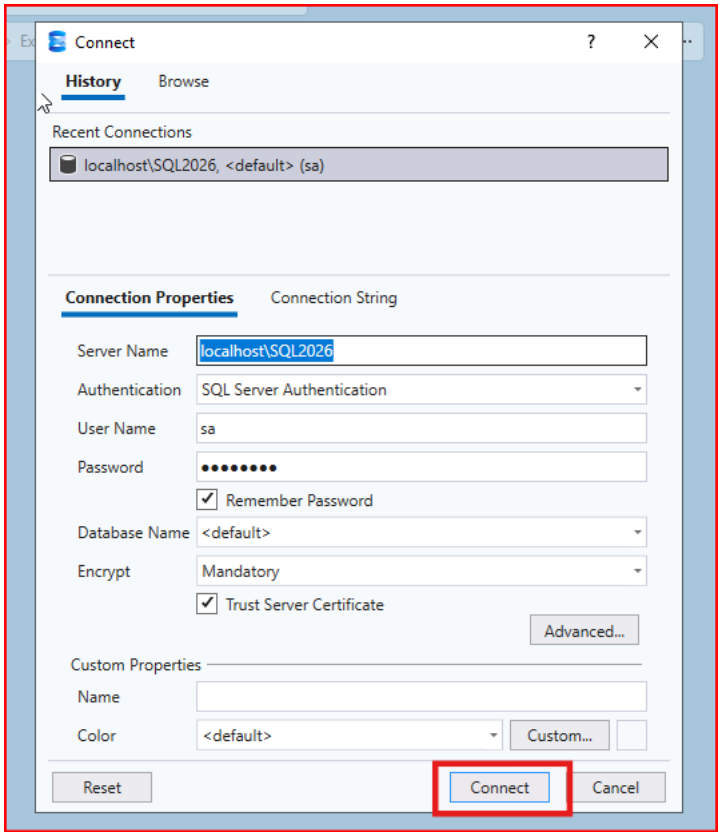
Objective: Encrypt a column in the existing empdata table and confirm it's unreadable in plain form.

Key Result: When viewing the table directly in SQL Server Management Studio, the encrypted column shows unreadable data. Only authorized client applications can decrypt and display the original values. This provides strong column-level protection.

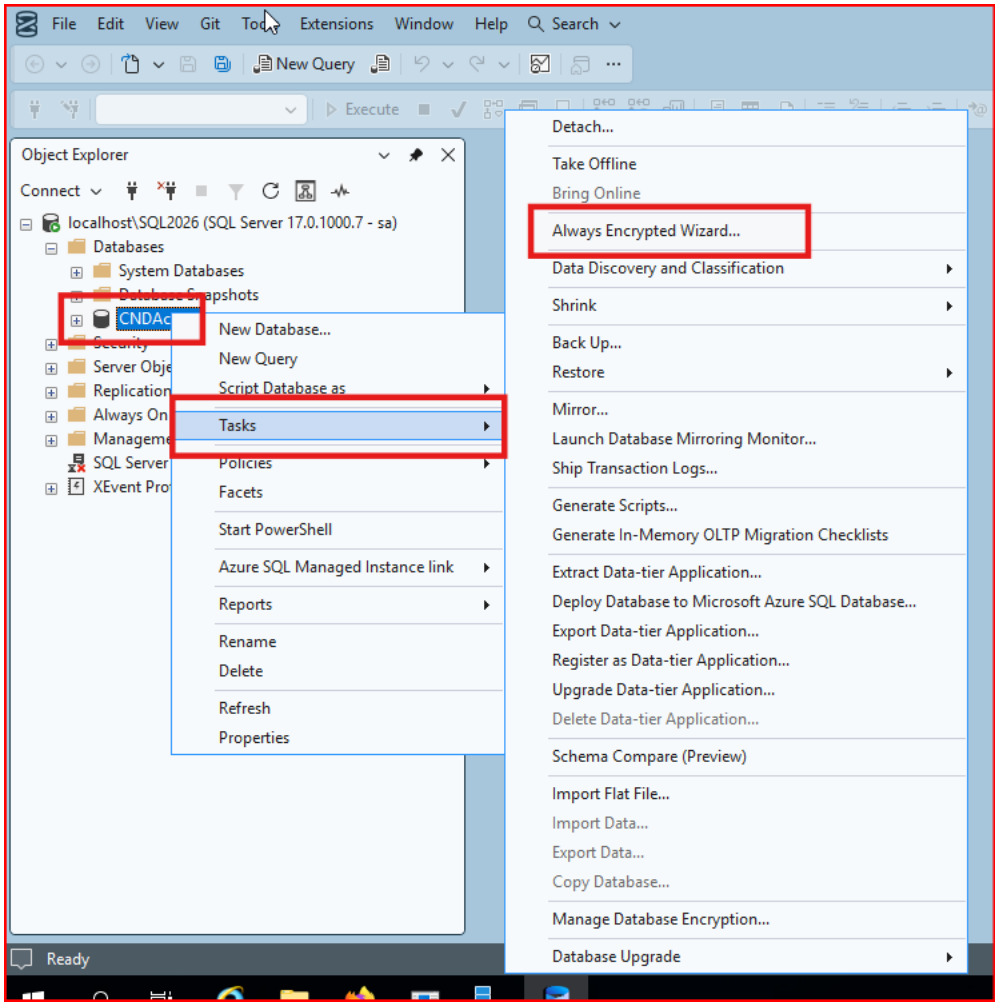
Open SSMS (SQL Server Management Studio)



Connect to the server



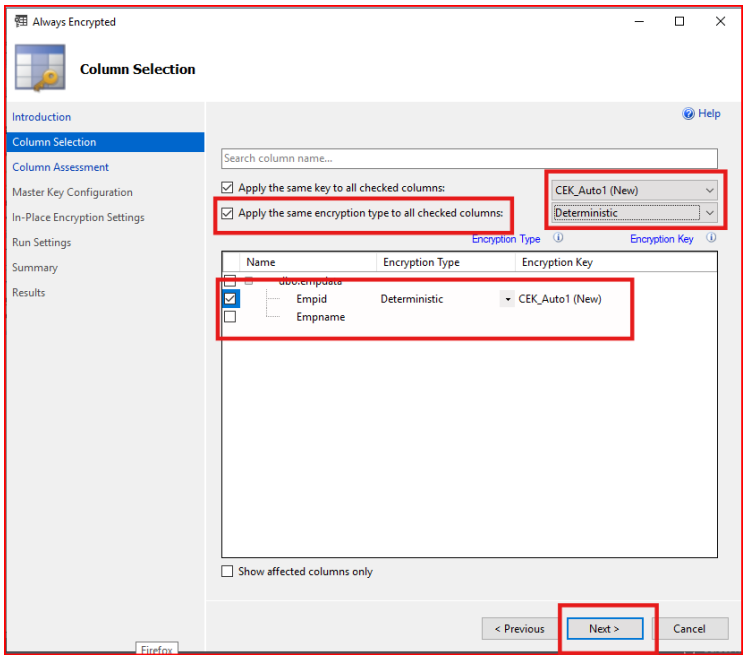
Right Click on CNDAccess -> Select Tasks-> Select Wizard



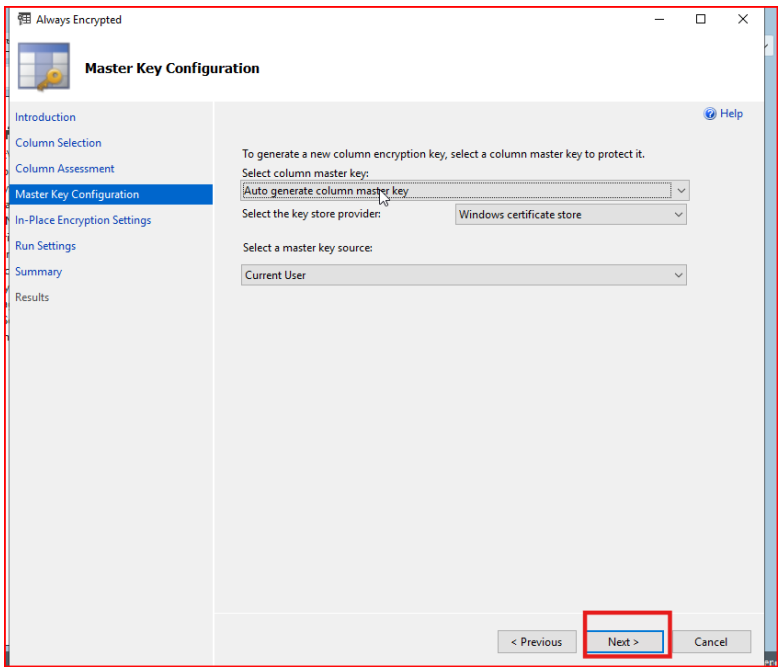
Create Column Master Key

On the **Introduction** page, click **Next**.

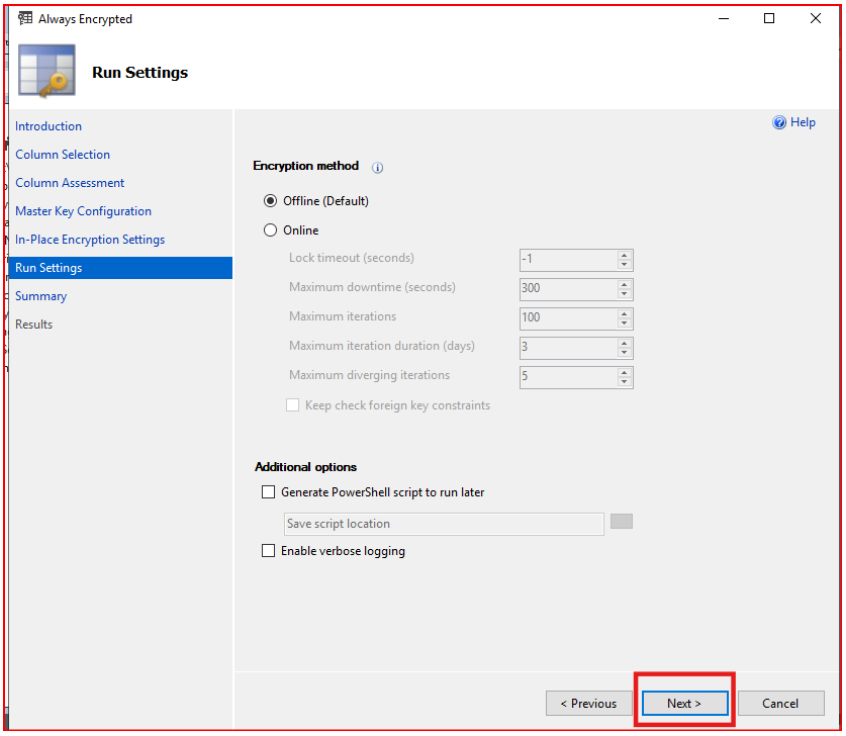
On **Column Selection**: check the **Empid** column checkbox. Set **Encryption Type** to **Deterministic**. Leave **Encryption Key** as **CEK_Auto1 (New)**. Click **Next**.



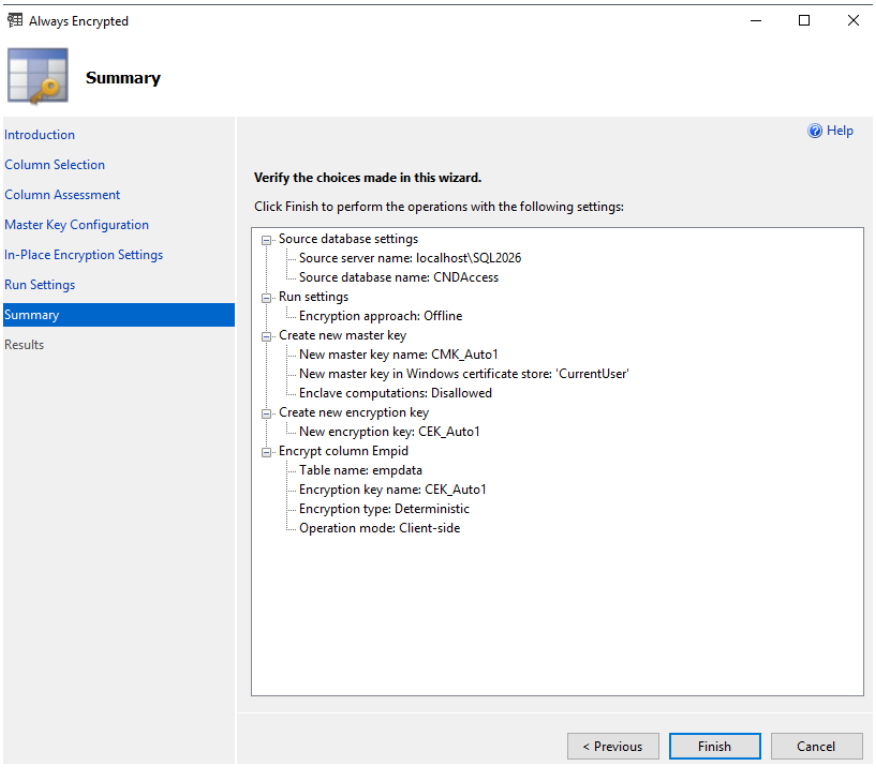
On **Master Key Configuration**: leave everything default (Auto generate column master key, Windows certificate store, Current User). Click **Next**.



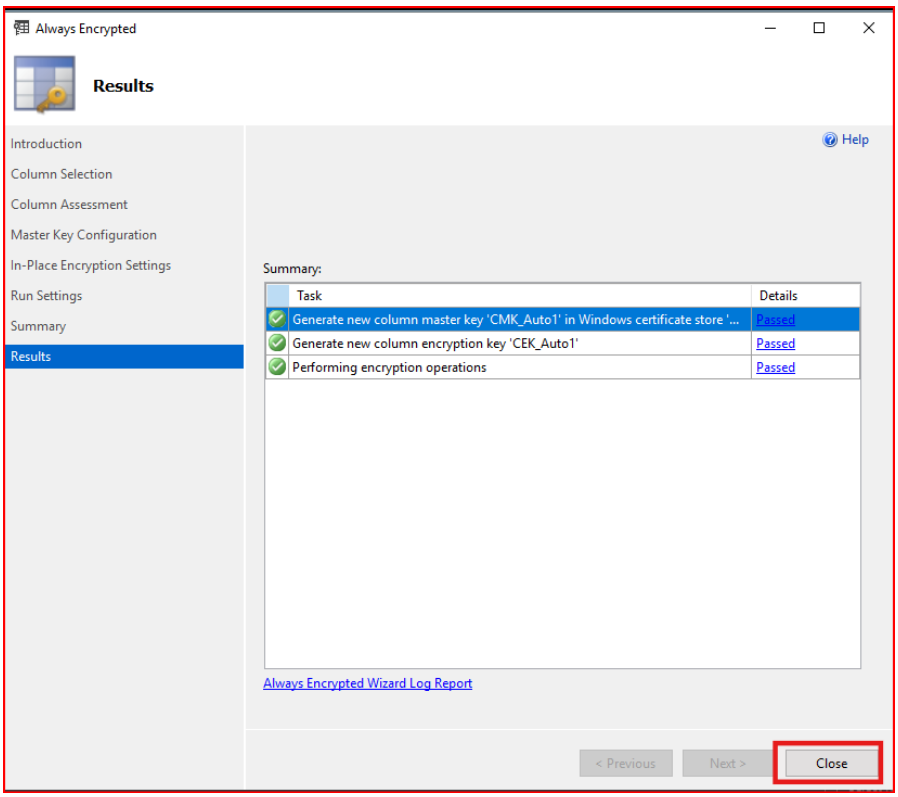
On **Run Settings**: Click **Next**.



On **Summary**: review the listed settings, click **Finish**.

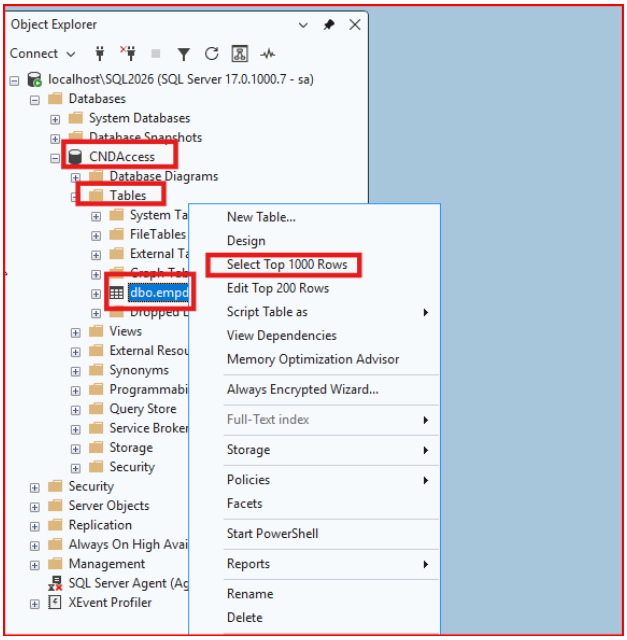


Wait for it to process — you'll see a **Results** page with tasks marked **Passed**. Click **Close**.



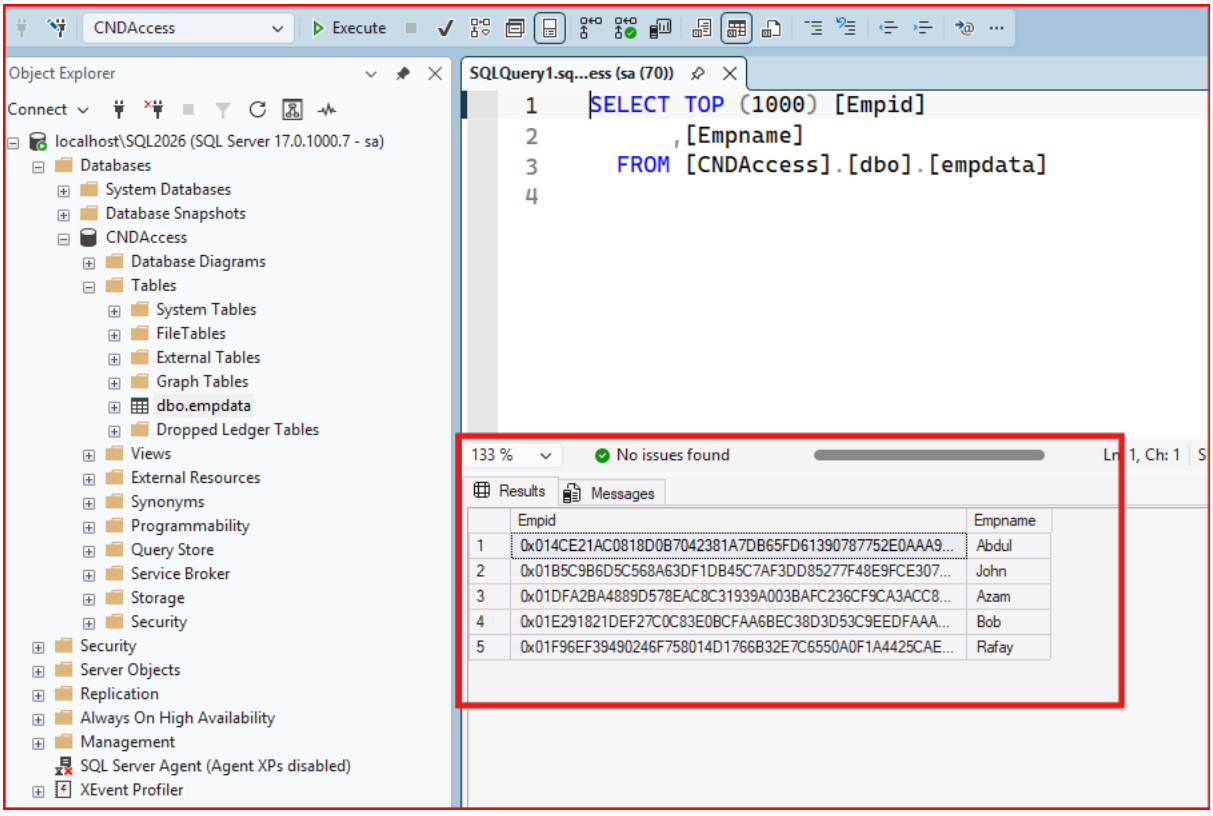
In Object Explorer, expand **CNDAccess → Tables**.

Right-click **dbo.empdata → Select Top 1000 Rows**.



Verify Column is Encrypted (Data appears unreadable)

Check the output — **Empid** column should now show long encrypted values (e.g., 0x01210B169F...) instead of plain numbers, while **Empname** remains readable.



What's Actually Happening?

- In normal SQL Server, when you store data in a table, the database engine can see the actual values (plain text). Even a database administrator with full access can view sensitive information by simply querying the table.
- **Always Encrypted** changes this completely. It encrypts the selected column (in this case, Empid) **on the client side** before the data is sent to SQL Server.
- The encryption keys (Column Master Key and Column Encryption Key) are stored on the client machine (or in a secure store), **not** inside the SQL Server database engine.

As a result:

  - When you open the table in SSMS, the encrypted column shows long binary values (like 0x01210B169F...) instead of the original numbers.
  - SQL Server itself never sees the real data — it only stores and works with the encrypted version.
  - Even a highly privileged database administrator cannot read the actual sensitive values.
- Only an authorized application that has access to the encryption keys can decrypt the data and show the original values to the user.

Conclusion – Module 10 (Part 1/3)

In this first part of Module 10, I successfully implemented three essential techniques for protecting **data at rest**:

- **Lab 1 (VeraCrypt):** Created an encrypted file container that remains completely unreadable when unmounted. This demonstrated strong file-level encryption for protecting sensitive documents.
- **Lab 2 (Transparent Data Encryption - TDE):** Enabled full database encryption on SQL Server. The entire database files and backups are now encrypted on disk, protecting them from theft or unauthorized access at the storage level.
- **Lab 3 (Always Encrypted):** Applied column-level encryption so that even database administrators cannot view sensitive data in plain text. The encryption keys remain on the client side, ensuring the database engine never sees the actual values.

Together, these three labs provided a strong practical foundation in modern data-at-rest security — from individual files to complete databases and even specific sensitive columns.

"This completes Part 1 of 3. Stay tuned for Part 2 covering SSL, PGP, and Secure Backup."